Instituto Profesional Duoc UC.
Escuela de Informática y Telecomunicaciones
Sede: Plaza Norte

“Signy”

Fecha: 08/11/2026

Informe presentado en conformidad a los requisitos para aprobar la asignatura Portafolio de Título

Autores:

Benjamín Andrés Vallejos Müller 21.524.820-8

Catalina Javiera Gutiérrez Vargas 21.273.766-6

Fernanda Javiera Poblete Llanos 21.357.764-6

Michael Ignacio Ibáñez Castro 20.581.284-9

Nombre del docente: Osnellys E, Andrade D.

Mes y año de presentación: noviembre de 2026

Santiago de Chile.

Índice

Contenido

Datos del documento 5

Información del Proyecto 6

Resumen 7

Abstract 7

Introducción 7

Avance y ajustes del Proyecto APT – Fase 2 8

Documentación del Proyecto 9

A. Análisis Preliminar del Caso 9

Modelado del proceso actual y del proceso propuesto 11

Proceso propuesto con Signy 12

B. Descubrimiento y visión del producto 14

Consideración lingüística y alcance territorial 17

C. Definición del producto 17

Product Goal 17

Visión del Proyecto – Visión y Cuatro Pilares 17

Visión del alcance – Impact Mapping 18

Definición Formal de entregables 18

D. Especificación de requisitos 21

Requisitos funcionales 21

Requisitos no funcionales 22

Historias de Usuario 23

Reglas de negocio 24

Restricciones y supuestos 24

E. Diseño de la solución 26

Arquitectura de software 26

Diagrama lógico de arquitectura 26

Vistas 4+1 de la arquitectura de Signy 27

Stack tecnológico 29

Modelo de datos 29

Diseño de interfaces / UX-UI 30

Diseño de seguridad 30

Integraciones 30

F. Diseño específico según naturaleza del proyecto 32

Componente IA / Visión Computacional 32

Métricas del componente 32

Validación 33
Limitaciones y justificación 33
Componentes especializados no aplicables 33
G. Planificación Ágil 34
Metodología de trabajo 34
Squad y responsabilidades 34
Épicas 34
Product Backlog priorizado 35
Criterio de priorización 35
User Story Mapping 35
Release Planning 36
H. Ejecución iterativa — Sprints 37
Sprint 1 – Base, seguridad y datos 37
Sprint 2 – Aprendizaje principal 38
Sprint 3 – Experiencia, comunidad y administración 39
Sprint 4 – Integración MediaPipe 40
Sprint 5 – Validación, personalización y cierre 40
Seguimiento y control de la gestión ágil 41
I. Estrategia y ejecución de pruebas 42
Plan de pruebas 42
Casos de prueba funcionales 42
Pruebas no funcionales 43
Gestión de defectos 44
J. Validación de la solución 46
Validación con usuarios y referentes 46
Cumplimiento de entregables 46
K. Evaluación económica y viabilidad 48
Recursos humanos 48
Recursos tecnológicos y operativos 48
Comparación de escenarios y selección del presupuesto 48
Viabilidad 49
L. Despliegue y operación 50
Arquitectura de despliegue 50
Ambientes 50
CI/CD 50
Configuración 50
Manual de instalación/despliegue 51
Manual de usuario / administrador 51
Plan básico de continuidad y mantenimiento 51

M. Resultados del proyecto 52
Producto final / estado actual 52
Resultados obtenidos 52
Indicadores y métricas 52
Proyección 53
Conclusión 53
Individual Conclusions 53
Fernanda Poblete – Product Owner 53
Catalina Gutiérrez – Scrum Master and Development Team 54
Benjamín Vallejos – Development Team 54
Michael Ibáñez – Development Team 54
Individual Reflections 54
Fernanda Poblete 54
Catalina Gutiérrez 54
Benjamín Vallejos 54
Michael Ibáñez 54
Referencias bibliográficas 55
Anexo de evidencias 59

Datos del documento

Versión	Fecha	Descripción/cambio	Autor
1.0.0	25/08/2026	Versión inicial. Desarrollo de Fase 1 - Definición del Proyecto APT: Datos del documento, Introducción, Análisis Preliminar del Caso, Descubrimiento y visión del producto, Definición del producto y Planificación inicial del proyecto.	Benjamín Vallejos, Catalina Gutiérrez, Fernanda Poblete y Michael Ibáñez
1.1.0	03/09/2026	Definición del alcance del proyecto, selección de herramientas tecnológicas a utilizar para el levantamiento de la aplicación, definición del cronograma.	Benjamín Vallejos, Catalina Gutiérrez, Fernanda Poblete y Michael Ibáñez
1.2.0	10/09/2026	Desarrollo y documentación de los ítems D a H: especificación de requisitos, diseño de la solución, componente de IA/visión computacional, planificación ágil y ejecución iterativa mediante Scrum.	Benjamín Vallejos, Catalina Gutiérrez, Fernanda Poblete y Michael Ibáñez
1.3.0	11/09/2026	Ajuste integral de Fase 2: incorporación de diagramas reales de arquitectura, integraciones, IA, backlog, user story mapping, release planning, roadmap de sprints y despliegue; reemplazo de contenido plantilla pendiente; revisión final de completitud del informe. Se incorporó evaluación comparativa de presupuestos bajo, medio y alto; selección del escenario medio; y diagramas AS IS, TO BE y arquitectura 4+1 con sus explicaciones.	Benjamín Vallejos, Catalina Gutiérrez, Fernanda Poblete y Michael Ibáñez
2.0.0	08/10/2026	Informe de avance Fase 2: resumen y abstract (ES/EN); sección de avance con evidencias verificadas y ajustes (dificultades, facilitadores y retroalimentación docente); numeración única de HU; Scrum Boards y release burndown actualizados; resultados reales de pruebas (607 unitarias y 34 de ML) y gestión de defectos; conclusiones y reflexiones individuales en inglés; formato Calibri 11 con numeración de páginas.	Benjamín Vallejos, Catalina Gutiérrez, Fernanda Poblete y Michael Ibáñez
3.0.0	08/11/2026	Actualización al estado final de la aplicación (rama v1): clasificador propio entrenado con TensorFlow en reemplazo de DTW; AI Studio con avatar 3D; Dashboard de KPIs; repaso priorizado por	Benjamín Vallejos, Catalina Gutiérrez, Fernanda Poblete y Michael Ibáñez

Versión	Fecha	Descripción/cambio	Autor
		fallos (HU-20); validación con la comunidad sorda y pruebas de usabilidad realizadas; señas equivalentes (Hola y Chao); aprobación desde 75 % con retroalimentación por niveles; resultados de pruebas actualizados (664 unitarias y 69 de ML); modelo de datos real (16 tablas); Scrum Boards, release burndown e indicadores finales.	

Información del Proyecto

Organización	Duoc UC. Escuela de Informática y Telecomunicaciones
Sección	002D
Proyecto (Nombre)	Signy - Plataforma de aprendizaje de Lengua de Señas Chilena (LSCh)
Fecha de Inicio	13/08/2026
Fecha de Término	Por definir según el calendario de la asignatura (cierre en Fase 3)
Patrocinador principal	Sin patrocinador externo formal; proyecto de iniciativa propia del equipo en el marco de la asignatura "Portafolio de Título". Colabora, sin aporte monetario, la Oficina de Inclusión de DIDECO de la Municipalidad de Huechuraba, que coordinó las reuniones con personas de la comunidad sorda para validar las señas. Se evalúa vincular a futuro a la Asociación de Sordos de Chile (ASOCH).
Docente	Osnellys Andrade

Resumen

Signy es una aplicación móvil que enseña Lengua de Señas Chilena (LSCh) a personas oyentes mediante lecciones gamificadas y práctica frente a la cámara. Este informe de la Fase 2 documenta la construcción del producto con Scrum: un Product Backlog de 20 historias de usuario distribuidas en cinco Sprints, una arquitectura móvil híbrida (Ionic/Angular con Capacitor) y Supabase como backend gestionado (autenticación, PostgreSQL con políticas RLS, Storage y Edge Functions). Al 8 de noviembre de 2026 se completaron las 20 historias de usuario y los 11 entregables comprometidos. El reconocimiento de señas funciona dentro de la lección: MediaPipe Hands detecta los puntos de las manos y un clasificador propio, una red neuronal entrenada con TensorFlow a partir de muestras grabadas en el AI Studio de la app, decide en el mismo celular qué seña se hizo. Las 12 señas de la app se validaron con personas de la comunidad sorda gracias a la coordinación de DIDECO Huechuraba, y la app fue probada por personas externas al equipo. La calidad se respalda con 664 pruebas unitarias de la aplicación (79 % de cobertura de sentencias) y 69 pruebas del pipeline de aprendizaje automático, todas aprobadas. El principal trabajo futuro es ampliar el vocabulario y el dataset con más personas para mejorar la exactitud del reconocimiento.

Palabras clave: Lengua de Señas Chilena, gamificación, aplicación móvil, MediaPipe, Scrum, Supabase.

Abstract

Signy is a mobile application that teaches Chilean Sign Language (LSCh) to hearing people through gamified lessons and camera-based practice. This Phase 2 report documents how the product was built with Scrum: a Product Backlog of 20 user stories distributed across five Sprints, a hybrid mobile architecture (Ionic/Angular with Capacitor) and Supabase as a managed backend (authentication, PostgreSQL with Row Level Security, Storage and Edge Functions). As of November 8, 2026, all 20 user stories and all 11 committed deliverables have been completed. Sign recognition runs inside the lesson: MediaPipe Hands detects the hand landmarks and the team's own classifier, a neural network trained with TensorFlow on samples recorded in the app's AI Studio, decides on the phone which sign was made. The 12 signs in the app were validated with members of the Deaf community through DIDECO Huechuraba, and people outside the team tested the application. Quality is supported by 664 unit tests of the application (79% statement coverage) and 69 tests of the machine learning pipeline, all passing. The main future work is to add more signs and more people to the dataset to improve recognition accuracy.

Keywords: Chilean Sign Language, gamification, mobile application, MediaPipe, Scrum, Supabase.

Introducción

La Ley N.° 21.303, promulgada en enero de 2021, reconoce a la Lengua de Señas Chilena (LSCh) como la lengua oficial de las personas sordas y establece la obligación del Estado de promover su uso en educación, salud y el ámbito laboral. Según el Segundo Estudio Nacional de la Discapacidad (ENDISC II, 2015), cuyos resultados difunde el Servicio Nacional de la Discapacidad (SENADIS), en Chile existen 712.005 personas con algún grado de pérdida auditiva, de las cuales se estima que 179.268 presentan sordera total. Sin embargo, a diferencia de lo que ocurre con los idiomas hablados donde existen plataformas gratuitas y gamificadas como Duolingo, en Chile no existe una herramienta digital equivalente, estructurada y de acceso masivo para aprender LSCh, lo que dificulta que familiares, docentes, intérpretes en formación y público general puedan acercarse a esta lengua de forma autónoma y progresiva.

Signy nace como respuesta a esta brecha: una aplicación móvil que enseña LSCh mediante un modelo de aprendizaje gamificado, inspirado en Duolingo, compuesto por lecciones con flashcards, ejercicios de

emparejamiento, cuestionarios y práctica frente a la cámara. El presente informe documenta, sustenta metodológicamente y da cuenta del avance de Signy como proyecto de ingeniería. Esta entrega de la Fase 2 amplía la definición inicial con la especificación de requisitos, el diseño de la solución, la planificación ágil, la ejecución de los Sprints, la estrategia y los resultados de las pruebas, la validación, la evaluación económica, el despliegue y los resultados obtenidos al 8 de noviembre de 2026.

El documento se organiza de la siguiente manera: luego del resumen y de esta introducción, la sección «Avance y ajustes del Proyecto APT – Fase 2» sintetiza el avance, las evidencias y los ajustes realizados. A continuación, la «Documentación del Proyecto» desarrolla los apartados A a M de la guía de la asignatura. Por último, se presentan la conclusión grupal, las conclusiones y reflexiones individuales (en inglés), las referencias bibliográficas y el anexo de evidencias.

Avance y ajustes del Proyecto APT – Fase 2

Durante la Fase 2 se ejecutaron los cinco Sprints. Al 8 de noviembre de 2026 se completaron las 20 historias de usuario y los 11 entregables. El reconocimiento de señas pasó por tres etapas: una heurística de píxeles, luego MediaPipe con comparación temporal (DTW, motor v3 del 04/10/2026) y, finalmente, un clasificador propio entrenado con TensorFlow (26/10/2026), que reemplazó a DTW porque reconocía mejor y hacía menos lento el celular. Las señas se validaron con personas de la comunidad sorda.

Evidencias de avance. Repositorio https://github.com/RatalinaPrograma/Proyecto-Senias (rama v1): código de la app (/signy), scripts SQL de la base de datos y Edge Functions (/supabase), pipeline de IA (/ml) y documentación de gestión (Documentos/FASE 2). Las pruebas se ejecutaron el 08/11/2026: 664 pruebas unitarias de la app aprobadas (1 omitida, 0 fallidas; 79,1 % de cobertura de sentencias) y 69 pruebas del pipeline de IA aprobadas.

Ajustes realizados. Cada ajuste se justifica por una dificultad, un facilitador o la retroalimentación docente de la Fase 1 (indicadores de relevancia, factibilidad, objetivos y evidencias evaluados como «Logrado»):

Ajuste	Origen	Justificación
Supabase y Edge Functions reemplazan el backend NodeJS (CC-001)	Facilitador	Supabase ya concentraba autenticación, datos y almacenamiento; reduce infraestructura propia.
Backlog consolidado en 20 HU con numeración única (CC-002)	Dificultad	Crecimiento del alcance; mejora la trazabilidad y la planificación.
Cinco Sprints: MediaPipe separado de la validación y el cierre (CC-003)	Dificultad	La integración técnica y la validación externa tienen riesgos distintos.
DIDECO Huechuraba como colaborador no monetario (CC-004)	Facilitador	Facilita el contacto con personas sordas para validar.
Fase limitada a Android; iOS como trabajo futuro (CC-005)	Dificultad / retroalimentación	Tiempo académico; responde a la observación sobre factibilidad.
Tres escenarios de presupuesto y elección del medio (CC-006)	Retroalimentación	Fundamenta la factibilidad económica del proyecto.
Objetivos reformulados como resultados observables	Retroalimentación	Responde a la observación sobre la precisión de los objetivos.
Cada evidencia indica qué demuestra y dónde verificarla	Retroalimentación	Responde a la observación sobre la justificación de las evidencias.

Ajuste	Origen	Justificación
Heurística de píxeles reemplazada por MediaPipe y un clasificador propio	Dificultad	La heurística no evaluaba la seña; DTW (motor v3) mejoró el resultado, pero el clasificador entrenado reconoce mejor y es más liviano.
Reglas de XP y permisos validadas en el servidor	Dificultad	Se detectó que podían alterarse desde el cliente.
AI Studio y Dashboard de KPIs en el panel de administración	Dificultad	Hacía falta grabar un dataset propio y medir el uso real de la app.
Señas grabadas con la comunidad sorda y validadas por ella	Facilitador	DIDECO Huechuraba coordinó las reuniones; el mayor obstáculo fue la demora en la comunicación.

Actividades pendientes o con retraso. No quedan historias ni entregables pendientes. Como mejora continua, el equipo seguirá grabando muestras de más personas para mejorar la exactitud del clasificador con usuarios nuevos (Michael y Catalina) y ampliará el vocabulario validado con la comunidad sorda (Fernanda).

Documentación del Proyecto

A. Análisis Preliminar del Caso.

Contexto organizacional: Signy es desarrollado por un equipo de cuatro estudiantes de Ingeniería en Informática de Duoc UC (Benjamín Vallejos, Catalina Gutiérrez, Fernanda Poblete y Michael Ibáñez), en el marco de la asignatura Capstone. No existe un mandante externo único: el proyecto se orienta a un escenario social real, la comunidad de personas oyentes que desea aprender Lengua de Señas Chilena (familiares y amigos de personas sordas, estudiantes, futuros intérpretes). El equipo trabajó junto a la Oficina de Inclusión de DIDECO de la Municipalidad de Huechuraba, que nos ayudó a relacionarnos con personas de la comunidad sorda para validar las señas de la app. Se proyecta, como colaborador natural, a organizaciones como la Asociación de Sordos de Chile (ASOCH) para ampliar el vocabulario validado.

Planteamiento del Problema: De acuerdo con el Segundo Estudio Nacional de la Discapacidad (ENDISC II, 2015), difundido por SENADIS, en Chile existen 712.005 personas con algún grado de pérdida auditiva, y se estima que 179.268 de ellas presentan sordera total. La Ley N.° 21.303 (2021) reconoce la LSCh como lengua oficial de las personas sordas y obliga a promover su uso en educación, salud y trabajo, pero esa obligación legal no viene acompañada de herramientas digitales masivas y accesibles para que la población oyente aprenda la lengua. Actualmente, quien quiere aprender LSCh en Chile debe recurrir a videos sueltos en YouTube, PDF's informales o cursos presenciales de pago, sin una ruta de aprendizaje progresiva, sin retroalimentación de práctica y sin seguimiento de avance. Esta dispersión de recursos genera una barrera de entrada que perpetúa la desconexión comunicativa entre personas sordas y oyentes, y limita la formación de intérpretes y profesionales de apoyo. El desafío que aborda Signy es, por tanto, la ausencia de una plataforma estructurada, gratuita y gamificada equivalente a lo que Duolingo representa para los idiomas hablados que permita aprender LSCh de forma autónoma y sostenida en el tiempo.

Levantamiento de información: Para esta fase se utilizaron: (a) revisión documental del marco legal vigente (Ley N.° 21.303 y Ley N.° 20.422) y de estadísticas oficiales de SENADIS; (b) benchmarking de aplicaciones existentes de aprendizaje de idiomas y de lengua de señas (Duolingo, SignSchool, Spread The Sign) para identificar patrones de gamificación aplicables; (c) revisión de la propuesta original del proyecto y del historial de commits del repositorio para verificar el estado real de avance del desarrollo (42 commits en la rama principal entre el 13/08/2026 y el 02/09/2026; al 08/11/2026 el repositorio registra 97 commits en cuatro ramas); (d) análisis interno del equipo sobre los recursos de LSCh disponibles públicamente en Chile; y (e) reuniones con personas de la comunidad sorda, coordinadas por DIDECO Huechuraba, en las que se grabaron y validaron las señas que hoy usa la app, en reemplazo de los íconos decorativos de las primeras versiones.

Modelado del proceso actual y del proceso propuesto

El diagrama AS IS representa la situación previa a la implementación de Signy. El aprendizaje de LSCh depende de recursos dispersos, como videos, redes sociales y apuntes, sin una ruta estructurada de contenidos ni un mecanismo de retroalimentación inmediata. El usuario debe observar e imitar las señas y posteriormente recurrir a terceros para comprobar si la ejecución es correcta. Esta dinámica dificulta medir el avance, mantener la continuidad de la práctica y detectar oportunamente los contenidos que requieren refuerzo.

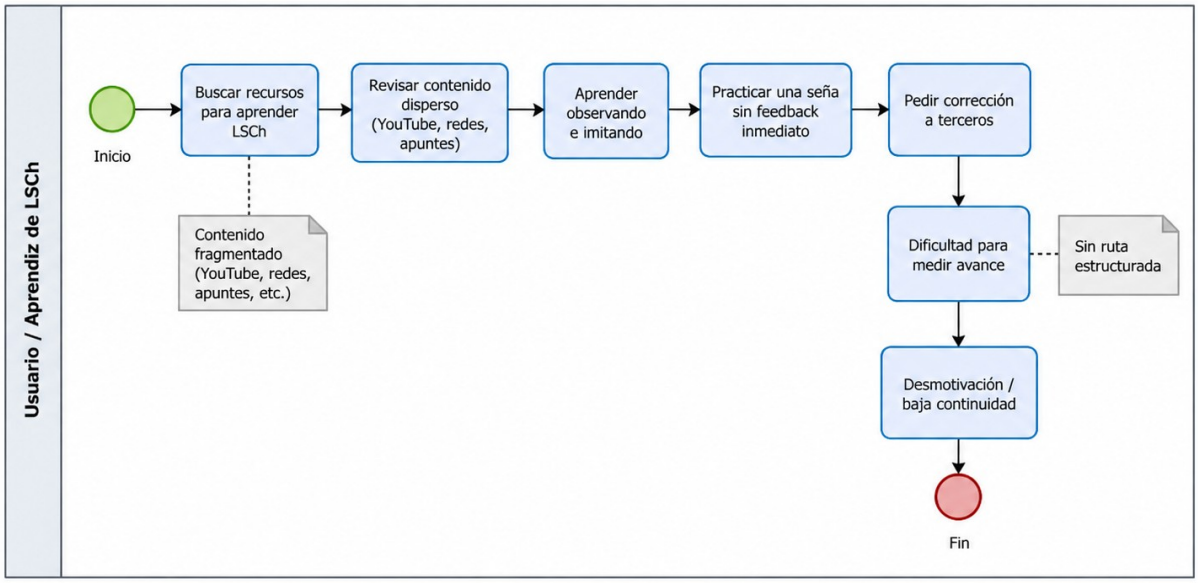


Figura 1. Diagrama AS IS del proceso de aprendizaje de LSCh antes de Signy.
Fuente: elaboración propia del equipo Signy (2026), a partir del levantamiento del proceso actual y de las dificultades identificadas durante el análisis preliminar del caso.

Explicación: El AS IS permite visualizar la principal brecha que origina el proyecto: existe acceso a materiales, pero estos se encuentran fragmentados y no conforman un proceso continuo de aprendizaje. La ausencia de una ruta, seguimiento y feedback genera dificultad para medir el progreso y puede disminuir la continuidad del aprendizaje.

Proceso propuesto con Signy

El diagrama TO BE muestra cómo Signy transforma el proceso actual en una experiencia estructurada. El usuario accede a la aplicación, selecciona un nivel y subnivel, completa actividades y practica con la cámara. MediaPipe proporciona los landmarks de la mano y un clasificador propio estima qué seña se hizo y con qué confianza, para entregar retroalimentación, actualizar el progreso y almacenar la información en Supabase. La administración de contenido y la validación con la comunidad sorda complementan el proceso para asegurar la calidad del contenido.

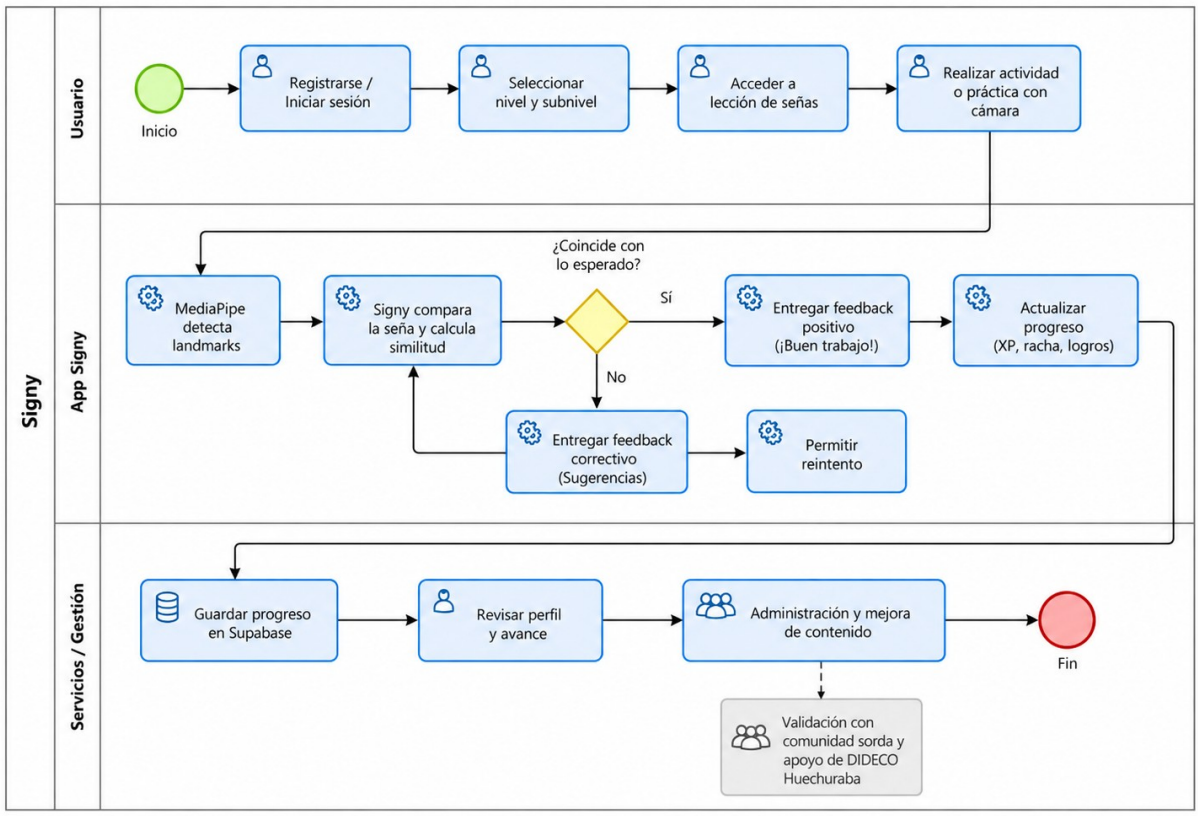


Figura 2. Diagrama TO BE del proceso de aprendizaje propuesto para Signy.
Fuente: elaboración propia del equipo Signy (2026), a partir de la arquitectura funcional, el flujo de aprendizaje y las actividades de validación definidas para el producto.

Explicación: El TO BE evidencia el valor agregado del producto: Signy concentra aprendizaje, práctica, retroalimentación, progreso y gestión de contenidos en una sola experiencia móvil. Además, incorpora una relación explícita entre el producto y la validación con la comunidad sorda, reforzando que la calidad lingüística debe acompañar al desarrollo tecnológico.

Justificación: Desde la perspectiva académica, Signy permite aplicar de forma integrada competencias del perfil de egreso de Ingeniería en Informática: arquitectura de aplicaciones móviles (Ionic/Angular con Capacitor), diseño y administración de bases de datos relacionales (Supabase/PostgreSQL), diseño de experiencia de usuario y visión computacional con aprendizaje automático (MediaPipe Hands y una red neuronal propia entrenada con TensorFlow) para reconocer señas. Desde la perspectiva profesional y social, el proyecto responde directamente al mandato de la Ley N.° 21.303 de promover el uso y aprendizaje de la LSCh, aportando una herramienta gratuita y escalable que puede reducir la brecha de

comunicación entre personas sordas y oyentes en Chile, y que puede constituir un antecedente reutilizable para instituciones educativas u organizaciones de la comunidad sorda.

Objetivo General: Desarrollar una aplicación móvil (Signy) que permita a personas oyentes aprender Lengua de Señas Chilena (LSCh) de forma autónoma, progresiva y gamificada, mediante un flujo de lecciones interactivas, un sistema de progreso y motivación, y práctica de señas asistida por cámara, sustentada en una capa de datos real en Supabase.

Objetivos específicos:

Implementar una capa de datos real en Supabase (autenticación, contenido de niveles/subniveles/señas y progreso del usuario) que reemplace por completo los almacenamientos locales usados en versiones tempranas.

Construir el flujo de lección interactivo de cuatro fases (flashcards, emparejamiento, cuestionario con ronda de repaso y práctica con cámara) para el contenido inicial definido por el equipo.

Diseñar e implementar el sistema de gamificación (vidas con regeneración basada en tiempo, racha y experiencia) que sostenga la motivación y el hábito de práctica diaria del usuario.

Incorporar reconocimiento de señas basado en MediaPipe Hands dentro de la lección, en reemplazo de la heurística de movimiento de píxeles usada en las primeras versiones, y registrar cada intento con su puntaje de similitud.

Establecer un proceso de validación de contenido con la comunidad sorda (idealmente junto a ASOCH) antes de reemplazar los íconos decorativos por señas lingüísticamente correctas.

Validar la usabilidad de la aplicación con usuarios representativos oyentes aprendiendo LSCh y, cuando sea posible, personas sordas antes del cierre del proyecto.

Alcance y exclusiones: Incluye: onboarding de bienvenida que se muestra solo en la primera apertura; registro, autenticación y recuperación de contraseña mediante código de seis dígitos enviado por correo desde una función serverless; autenticación de dos factores (2FA/TOTP) vía Supabase Auth MFA; contenido de niveles y subniveles almacenado y servido desde Supabase (hoy 4 niveles, 12 lecciones y 12 señas grabadas con la comunidad sorda), con estado «próximamente» y guard de acceso para los subniveles sin contenido; flujo de lección de cuatro fases (flashcards, emparejamiento con señas reales y señas «señuelo», cuestionario con ronda de repaso y práctica con cámara) con texto a voz integrado y barra de carga real; reconocimiento de señas en el celular con MediaPipe Hands y un clasificador propio, que aprueba la seña desde 75 % de seguridad y entrega retroalimentación según el resultado (consejos reales para mejorar o, al 100 %, una felicitación); sistema de gamificación (vidas, racha con congeladores,

experiencia, logros y ranking entre amigos) con persistencia real y calendario de racha de los últimos 28 días en el perfil; repaso de señas anteriores y lista de «señas por reforzar» según los fallos del usuario; notificaciones locales configurables (recordatorio diario y aviso cuando se recupera una vida); perfil de usuario con estadísticas y sistema social (buscar usuarios, seguidores, seguidos y regalo de congeladores); panel de administración con gestión de vocabulario (GIF, WebP, imagen y video MP4), AI Studio para grabar el dataset de la IA con un avatar 3D de referencia y Dashboard de KPIs de uso; caché permanente en disco de los recursos multimedia dentro del WebView, con precarga en segundo plano; y seguimiento de progreso por nivel y subnivel.

No incluye por ahora: certificación formal del contenido por un organismo oficial (las 12 señas actuales se validaron con personas de la comunidad sorda, pero no cuentan con una certificación institucional); un clasificador entrenado con un dataset amplio y diverso (el modelo actual se entrenó con 169 muestras y su exactitud con personas que no participaron en el entrenamiento todavía es baja); funcionamiento completo sin conexión de los datos de la aplicación (la caché permanente cubre los archivos multimedia y el clasificador viaja dentro de la app); el módulo de backend/API en NodeJS planteado en la propuesta original, cuya lógica vive hoy como funciones serverless en Supabase Edge Functions (delete-account y password-reset); despliegue en tiendas de aplicaciones (Play Store) y monetización.

Trabajo futuro: ampliación del vocabulario validado junto a la comunidad sorda, ASOCH o intérpretes certificados; más muestras de más personas para mejorar la exactitud del clasificador; reconocimiento de expresiones no manuales; extensión de la caché permanente a los datos de la aplicación para alcanzar un modo sin conexión completo; publicación en Play Store; migración de tipos manuales (db-types.ts) a tipos autogenerados con el comando supabase gen types typescript.

B. Descubrimiento y visión del producto

Mapa Mental

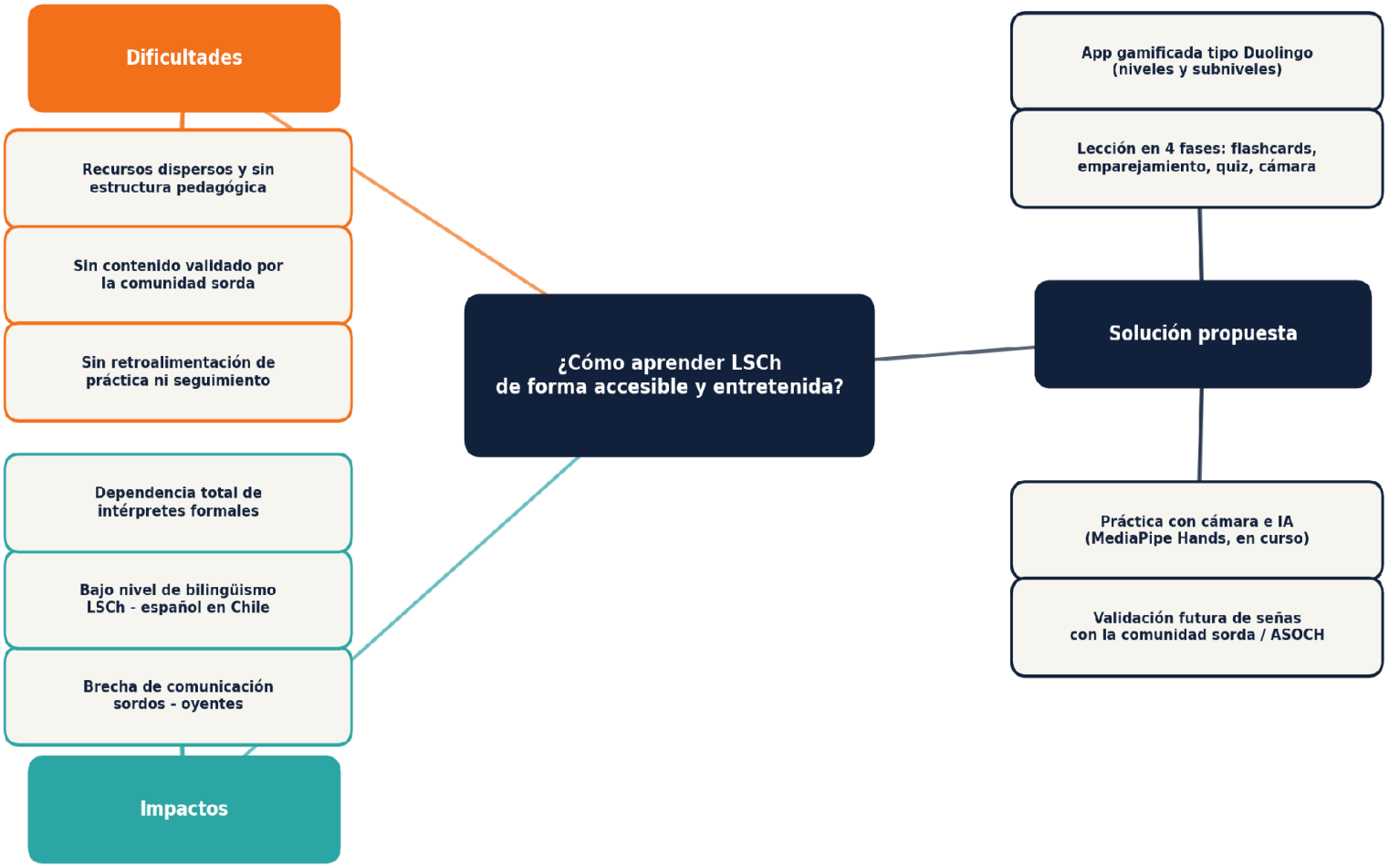


Figura 3. Mapa mental del problema abordado por Signy: dificultades detectadas, impactos y solución propuesta.

Fuente: elaboración propia del equipo Signy (2026), diagramada con Python y la biblioteca Matplotlib. El contenido se basa en el análisis del problema desarrollado en este informe, sustentado en la Ley N.° 21.303 (2021) y en los datos del Segundo Estudio Nacional de la Discapacidad (ENDISC II, 2015) difundidos por SENADIS.

Mapa de actores:

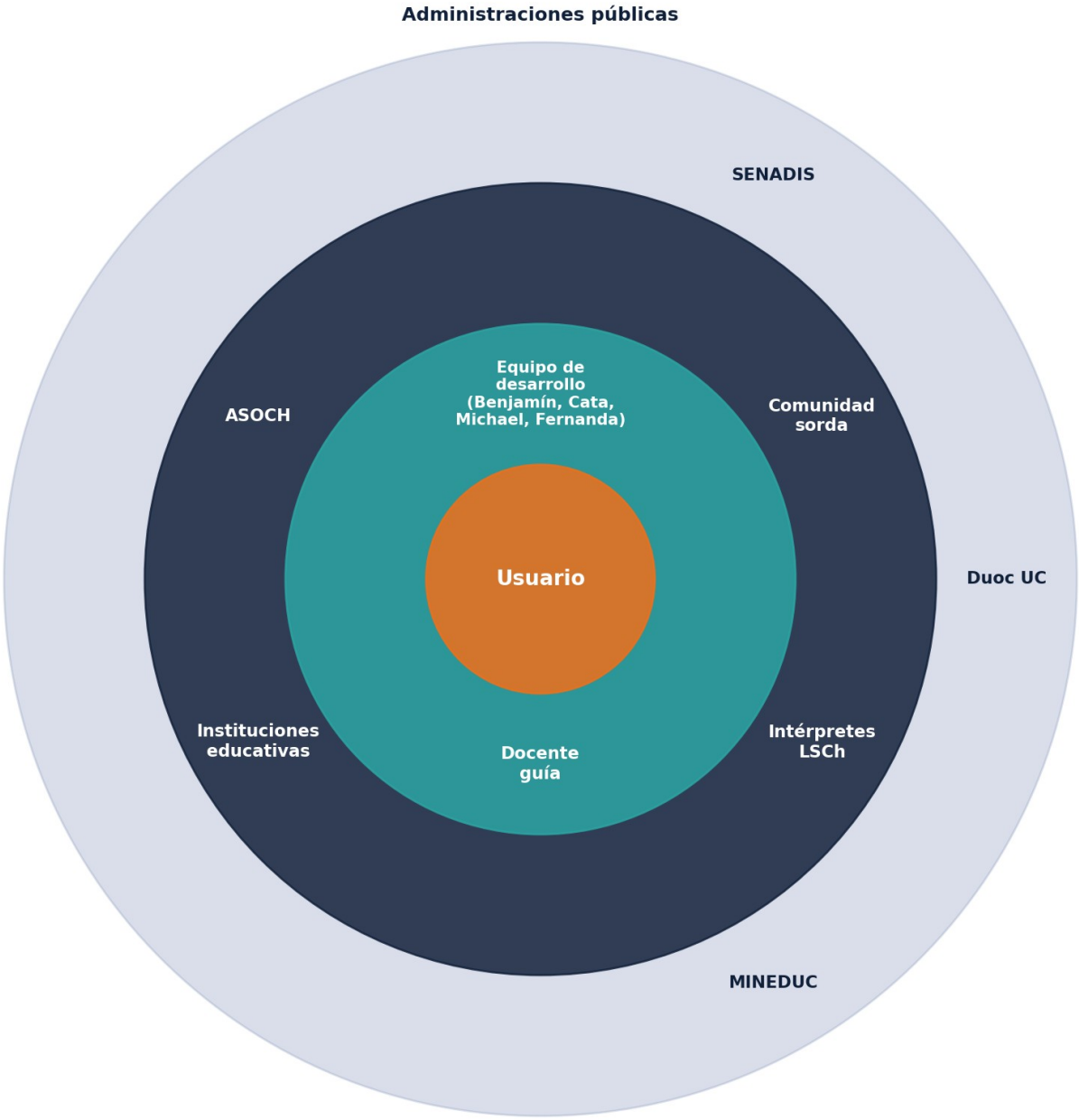


Figura 4. Mapa de actores de Signy, desde el usuario hasta las administraciones públicas vinculadas a la inclusión de personas sordas en Chile.
Fuente: elaboración propia del equipo Signy (2026), diagramada con Python y la biblioteca Matplotlib, a partir del levantamiento de actores realizado por el equipo y del marco institucional de la Ley N.° 21.303 (2021).

Necesidades de los usuarios:

Actor	Necesidad	Problema actual	Expectativa
Persona oyente que quiere aprender LSCh	Aprender vocabulario y gramática básica	Recursos dispersos, sin estructura pedagógica	App con lecciones cortas y gamificadas, disponible en el celular, con

(familiar, amigo, estudiante)	de LSCh de forma progresiva y entretenida	ni retroalimentación de práctica	seguimiento de progreso
Persona sorda / comunidad sorda	Que el contenido enseñado sea lingüísticamente correcto y respete su cultura	Apps y videos existentes muchas veces usan señas inventadas o mal ejecutadas	Validación del contenido junto a hablantes nativos de LSCh antes de su publicación
Docente / intérprete LSCh	Contar con una herramienta de apoyo para reforzar el aprendizaje de sus estudiantes	No existe una plataforma chilena equivalente a Duolingo para LSCh	Plataforma confiable que complemente la enseñanza presencial

Consideración lingüística y alcance territorial

Las lenguas de señas no constituyen un idioma universal. Cada comunidad sorda desarrolla lenguas naturales propias, con vocabulario, gramática y variaciones territoriales. En Chile, la Lengua de Señas Chilena (LSCh) es la lengua natural de la comunidad sorda del país y presenta incluso variaciones regionales, del mismo modo que ocurre en las lenguas orales. Por esta razón, Signy se delimita específicamente a LSCh y no debe reutilizar automáticamente vocabulario de otras lenguas de señas, como ASL (Estados Unidos) o LSE (España). Esta decisión refuerza la necesidad de validar el contenido con personas sordas chilenas, intérpretes certificados u organizaciones especializadas antes de presentarlo como material lingüísticamente validado (Acuña Robertson et al., 2009).

C. Definición del producto

Product Goal

Convertir a Signy en la plataforma de referencia para el aprendizaje autónomo y gamificado de la Lengua de Señas Chilena, ofreciendo una experiencia tan accesible y motivante como aprender un idioma hablado en Duolingo, y respaldada, en su versión madura, por contenido validado junto a la comunidad sorda.

Visión del Proyecto – Visión y Cuatro Pilares

Signy se sostiene en cuatro pilares. Valor para el usuario: aprendizaje entretenido, gratuito y accesible desde el celular, con progreso medible y motivación constante. Factibilidad técnica: un stack ya validado por el equipo, Ionic/Angular con Capacitor, Supabase como backend gestionado (autenticación, base de datos y funciones serverless), MediaPipe Hands y un clasificador propio entrenado con TensorFlow que funciona dentro del celular, sobre los cuales se construyeron módulos completos de autenticación con 2FA, perfil social, gamificación con racha y notificaciones, administración de contenido, AI Studio y Dashboard de KPIs. Impacto organizacional y social: aporte concreto al mandato de inclusión de la Ley N.° 21.303 y acercamiento entre la comunidad sorda y oyente, con señas validadas junto a personas sordas. Sostenibilidad del desarrollo: arquitectura modular por fases de lección, backend gestionado que

reduce la carga operativa, pipeline de IA reproducible y migración planificada hacia tipos autogenerados de Supabase para reducir deuda técnica.

Visión del alcance – Impact Mapping

El alcance se delimita mediante Impact Mapping (Adzic, 2012), técnica que relaciona actores, impactos deseados y entregables para fijar las fronteras del proyecto.

Actor	Impacto deseado	Entregable relacionado
Usuario oyente aprendiz	Aprender vocabulario y gramática de LSCh de forma progresiva	Flujo de lección de 4 fases + sistema de vidas, racha y progreso
Persona sorda / comunidad sorda	Sentir que el contenido representa correctamente su lengua y cultura	Validación de las señas con personas de la comunidad sorda (realizada con apoyo de DIDECO Huechuraba)
Docente / intérprete LSCh	Contar con una herramienta confiable de apoyo o referencia	Contenido estructurado por niveles, disponible como material de refuerzo

Meta (why): facilitar el aprendizaje accesible y gamificado de la Lengua de Señas Chilena.

Definición Formal de entregables

N.°	Entregable	Descripción	Criterio de aceptación
1	Autenticación, sesión y seguridad de cuenta	Registro, login, recuperación de contraseña y autenticación de dos factores (2FA/TOTP) vía Supabase Auth.	El usuario puede registrarse, iniciar sesión, recuperar su contraseña y activar 2FA. (Implementado).
2	Capa de datos real en Supabase	Niveles, subniveles, señas y progreso servidos desde PostgreSQL, reemplazando almacenamiento local.	Todo el contenido y el progreso se leen y escriben desde la base de datos. (Implementado).
3	Flujo de lección interactivo (4 fases) con texto a voz	Flashcards, emparejamiento, cuestionario con ronda de repaso y práctica con cámara; lectura en voz alta opcional del vocabulario.	El usuario completa las 4 fases de una lección, escucha el vocabulario si lo activa, y su avance queda registrado. (Implementado).
4	Sistema de gamificación	Vidas con regeneración por	Las vidas se descuentan y regeneran correctamente y se reflejan en la

		tiempo (1 cada 4 h, máx. 5), racha y experiencia (XP).	interfaz. (Implementado).
5	Perfil de usuario y sistema social	Perfil con estadísticas, búsqueda de usuarios, seguidores y seguidos.	El usuario visualiza su perfil y puede seguir/dejar de seguir a otros usuarios. (Implementado).
6	Panel de administración de vocabulario	CRUD de niveles, subniveles y señas, con carga de multimedia (URL o subida directa), reservado a usuarios con `es_admin = true`; incluye AI Studio, para grabar el dataset de la IA, y un Dashboard de KPIs de uso.	Un usuario administrador puede crear, editar y eliminar contenido sin intervenir la base de datos manualmente. (Implementado).
7	Práctica con cámara y reconocimiento de señas	Captura con la cámara durante la lección; MediaPipe Hands (@mediapipe/tasks-vision) detecta los 21 puntos de cada mano y un clasificador propio (red neuronal entrenada con TensorFlow) reconoce la seña en el mismo celular.	El sistema captura el gesto del usuario y entrega retroalimentación. (Implementado: cámara en la lección desde el 25/09, motor v3 desde el 04/10 y clasificador propio desde el 26/10).
8	Validación lingüística de contenido con la comunidad sorda	Revisión del vocabulario junto a personas de la comunidad sorda y grabación de las señas con ellas, en reemplazo de los íconos decorativos iniciales.	El contenido publicado cuenta con revisión de al menos un hablante nativo o referente de la comunidad sorda. (Implementado: reuniones coordinadas por DIDECO Huechuraba; las 12 señas de la app se grabaron con la comunidad).
9	Notificaciones locales	Recordatorio diario a la hora que elija el usuario y aviso cuando se recupera una vida, con mensajes rotativos y permisos gestionados desde Ajustes. Funciona	El usuario activa, configura y recibe las notificaciones desde su teléfono. (Implementado).

		en el propio dispositivo, sin servidor de notificaciones push.	
10	Caché permanente de multimedia y carga con progreso	Caché en disco (signy-media-v1) que descarga cada recurso una sola vez, precarga en segundo plano y barra de carga real en la lección; soporte de GIF, WebP, imagen y video MP4.	Al repetir un subnivel ya visto, los recursos se leen desde el dispositivo sin volver a descargarse de Supabase Storage. (Implementado).
11	Onboarding y control del contenido disponible	Pantallas de bienvenida que aparecen solo en la primera apertura y marca «próximamente» para los subniveles que aún no tienen contenido, con un guard que impide entrar a lecciones vacías.	El usuario nuevo ve el onboarding una única vez y no puede iniciar subniveles sin contenido. (Implementado).

D. Especificación de requisitos

Requisitos funcionales

ID	Requisito	Prioridad	Entregable asociado
RF-01	Registrar usuarios mediante correo electrónico y contraseña.	Alta	Autenticación
RF-02	Iniciar y cerrar sesión de manera segura.	Alta	Autenticación
RF-03	Recuperar contraseña mediante código enviado al correo registrado.	Alta	Autenticación
RF-04	Activar autenticación de dos factores (2FA/TOTP).	Media	Seguridad de cuenta
RF-05	Mostrar niveles y subniveles de aprendizaje.	Alta	Ruta de aprendizaje
RF-06	Controlar acceso a contenido disponible, bloqueado o “próximamente”.	Alta	Control de contenido
RF-07	Obtener niveles, subniveles, señas y progreso desde Supabase.	Alta	Datos en Supabase
RF-08	Ejecutar lecciones con flashcards, emparejamiento, quiz y cámara.	Alta	Flujo de lección
RF-09	Generar una ronda de repaso ante respuestas incorrectas.	Alta	Flujo de lección
RF-10	Reproducir vocabulario mediante texto a voz.	Media	Accesibilidad / lección
RF-11	Registrar progreso por nivel y subnivel.	Alta	Progreso
RF-12	Administrar hasta cinco vidas y regenerar una cada cuatro horas.	Alta	Gamificación
RF-13	Registrar y actualizar racha de práctica y XP.	Alta	Gamificación
RF-14	Aplicar mecanismos de protección de racha.	Media	Gamificación
RF-15	Mostrar estadísticas de avance, experiencia y racha.	Media	Perfil
RF-16	Buscar usuarios, seguirlos y dejar de seguirlos.	Media	Sistema social

ID	Requisito	Prioridad	Entregable asociado
RF-17	Permitir CRUD de niveles, subniveles y señas a administradores.	Media	Administración
RF-18	Permitir carga de multimedia mediante URL o subida directa.	Media	Administración
RF-19	Capturar la ejecución de una seña mediante cámara.	Alta	Práctica con cámara
RF-20	Detectar landmarks de las manos con MediaPipe Hands.	Alta	Visión computacional
RF-21	Entregar retroalimentación sobre la ejecución de una seña.	Alta	Reconocimiento
RF-22	Almacenar en caché recursos multimedia descargados.	Alta	Rendimiento
RF-23	Mostrar progreso de carga de recursos de la lección.	Media	UX / rendimiento
RF-24	Configurar recordatorios locales de práctica.	Media	Notificaciones
RF-25	Notificar cuando una vida haya sido recuperada.	Media	Notificaciones
RF-26	Mostrar onboarding solo durante la primera apertura.	Media	Onboarding
RF-27	Visualizar y actualizar información básica del perfil.	Media	Perfil
RF-28	Registrar intentos, aciertos, errores y score de similitud cuando corresponda.	Media	Analítica de aprendizaje
RF-29	Desbloquear logros o insignias al cumplir condiciones de progreso.	Media	Logros
RF-30	Priorizar para repaso las señas con mayor cantidad de fallos del usuario.	Media	Repaso inteligente

Requisitos no funcionales

ID	Categoría	Requisito
RNF-01	Seguridad	Supabase Auth deberá gestionar sesiones de manera segura.

Historias de Usuario

ID	Historia de usuario
HU-01	Como usuario, quiero registrarme e iniciar sesión, para almacenar mi progreso.
HU-02	Como usuario, quiero recuperar mi contraseña, para volver a acceder a mi cuenta.
HU-03	Como usuario, quiero activar 2FA, para aumentar la seguridad de mi cuenta.
HU-04	Como usuario, quiero ver niveles y subniveles, para seguir una ruta de aprendizaje.
HU-05	Como usuario, quiero completar lecciones con distintos ejercicios, para aprender LSCh de forma dinámica.
HU-06	Como usuario, quiero repasar mis errores, para reforzar el contenido que más me cuesta.
HU-07	Como usuario, quiero visualizar mi progreso, para conocer cuánto he avanzado.
HU-08	Como usuario, quiero disponer de vidas que se regeneren, para contar con una mecánica de desafío.
HU-09	Como usuario, quiero obtener XP y mantener una racha, para motivarme a practicar.
HU-10	Como usuario, quiero consultar mi perfil y estadísticas, para conocer mi rendimiento.
HU-11	Como usuario, quiero buscar y seguir a otros usuarios, para incorporar interacción social.
HU-12	Como administrador, quiero gestionar contenido, para actualizar vocabulario sin editar directamente la BD.
HU-13	Como usuario, quiero practicar frente a la cámara, para recibir retroalimentación.
HU-14	Como usuario, quiero que se analicen landmarks de mis manos, para obtener una evaluación más precisa.
HU-15	Como usuario, quiero configurar recordatorios, para mantener mi hábito de práctica.
HU-16	Como usuario, quiero saber cuándo recupero una vida, para continuar aprendiendo.
HU-17	Como usuario nuevo, quiero recibir una introducción, para comprender el funcionamiento de Signy.
HU-18	Como usuario, quiero que las lecciones ya visitadas carguen rápido, para reducir tiempo y consumo de datos.
HU-19	Como usuario, quiero obtener logros o insignias, para reconocer hitos importantes de mi progreso.
HU-20	Como usuario, quiero un repaso priorizado según mis fallos, para concentrarme en las señas que más necesito practicar.

Reglas de negocio

ID	Regla de negocio
RN-01	El usuario podrá mantener un máximo de cinco vidas.
RN-02	Se regenerará una vida cada cuatro horas hasta alcanzar el máximo.
RN-03	La racha se actualizará según la práctica registrada.
RN-04	El progreso se almacenará individualmente por usuario y contenido.
RN-05	No se permitirá ingresar a subniveles sin contenido habilitado.
RN-06	Los subniveles sin contenido se identificarán como “próximamente”.
RN-07	Solo usuarios autorizados administrarán contenido.
RN-08	El onboarding aparecerá una única vez por instalación/usuario según la implementación.
RN-09	La multimedia descargada se reutilizará desde caché cuando esté disponible.
RN-10	Progreso, estadísticas y configuraciones quedarán vinculados al usuario autenticado.
RN-11	Los intentos podrán registrar resultado y similitud cuando exista reconocimiento.
RN-12	El contenido presentado como validado deberá contar con revisión lingüística competente.
RN-13	MediaPipe no equivale por sí solo a validación lingüística de LSCh.
RN-14	Las notificaciones dependerán de permisos del dispositivo.
RN-15	Los logros y repasos personalizados se calcularán a partir de actividad real registrada del usuario.

Restricciones y supuestos

ID	Restricción / supuesto	Por qué
R-01	Tiempo académico limitado	El alcance debe priorizar valor porque el proyecto debe completarse dentro del semestre.
R-02	Dependencia de Internet	Autenticación, datos y progreso dependen de Supabase; actualmente la caché cubre principalmente multimedia.
R-03	Cámara y permisos	La práctica visual requiere cámara disponible y autorización del usuario.
R-04	Condiciones de captura	Iluminación, distancia, oclusiones y calidad de cámara pueden alterar la detección de landmarks.
R-05	Variación lingüística	Una seña no puede validarse únicamente por similitud geométrica; la LSCh exige revisión lingüística y cultural.

ID	Restricción / supuesto	Por qué
R-06	Contenido validado	El contenido definitivo depende de participación de comunidad sorda, intérpretes o referentes especializados.
R-07	Plataforma móvil objetivo: Android	Se prioriza Android por factibilidad académica, disponibilidad de herramientas/equipos de prueba y uso de Capacitor/Android Studio. Aunque Ionic/Angular es multiplataforma, iOS queda fuera de la validación actual porque requeriría un entorno macOS/Xcode y pruebas específicas adicionales.
R-08	Backend gestionado	La propuesta inicial de NodeJS independiente se simplificó utilizando Supabase y Edge Functions para reducir carga operativa.
R-09	Sin publicación en tiendas	Play Store/App Store y monetización quedan fuera del alcance académico actual para evitar costos y procesos de publicación que no aportan al objetivo principal de la fase.
R-10	No universalidad de la lengua de señas	Signy trabaja con LSCh porque las lenguas de señas varían entre países y comunidades; no corresponde importar automáticamente señas de ASL, LSE u otras lenguas.

E. Diseño de la solución

Arquitectura de software

Signy utiliza una arquitectura móvil híbrida basada en capas. La interfaz y la lógica de presentación se desarrollan con Ionic y Angular; Capacitor conecta la aplicación con capacidades nativas del dispositivo; los servicios Angular coordinan autenticación, progreso, gamificación, recursos y comunicación con Supabase; PostgreSQL almacena los datos relacionales; Supabase Storage conserva multimedia; Edge Functions resuelve procesos serverless; y MediaPipe Hands procesa la cámara para extraer landmarks. Esta separación busca mantener la solución modular, escalable y fácil de mantener.

Diagrama lógico de arquitectura

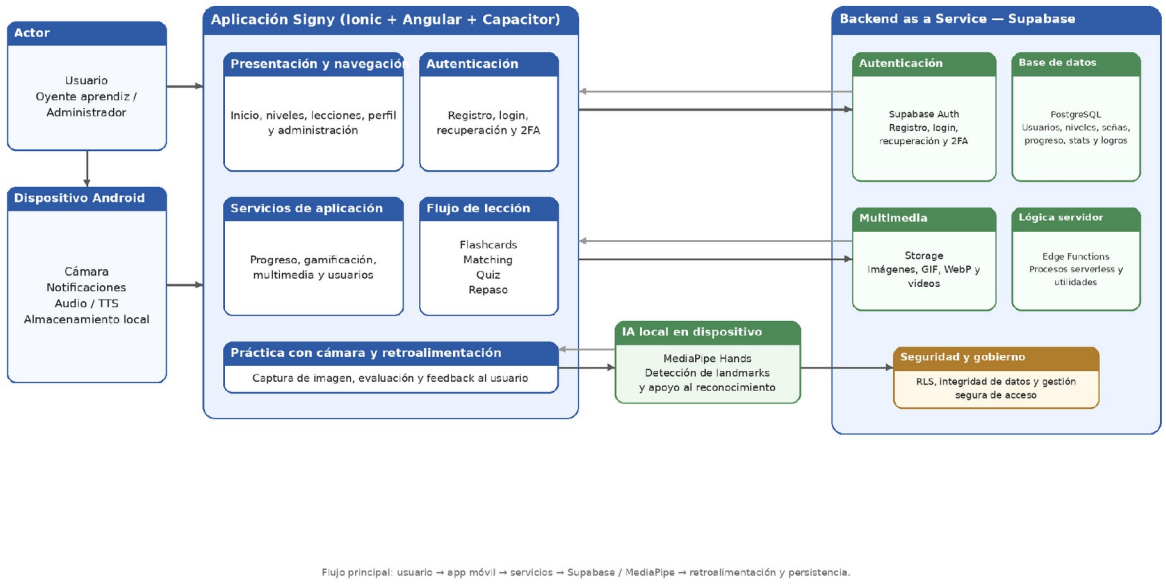


Figura 5. Diagrama lógico de arquitectura de Signy. Fuente: elaboración propia del equipo Signy (2026).

Vistas 4+1 de la arquitectura de Signy

Para complementar el diagrama lógico de arquitectura, el modelo 4+1 (Kruchten, 1995) permite observar Signy desde distintas perspectivas. La vista lógica representa los componentes funcionales y el recorrido del usuario; la vista de desarrollo organiza las tecnologías y módulos que construyen la solución; la vista de procesos muestra la secuencia de autenticación, aprendizaje, práctica, reconocimiento y persistencia; la vista física describe la ejecución sobre Android, la aplicación móvil y los servicios cloud; y la vista de escenarios relaciona estas perspectivas mediante los principales casos de uso del usuario y del administrador.

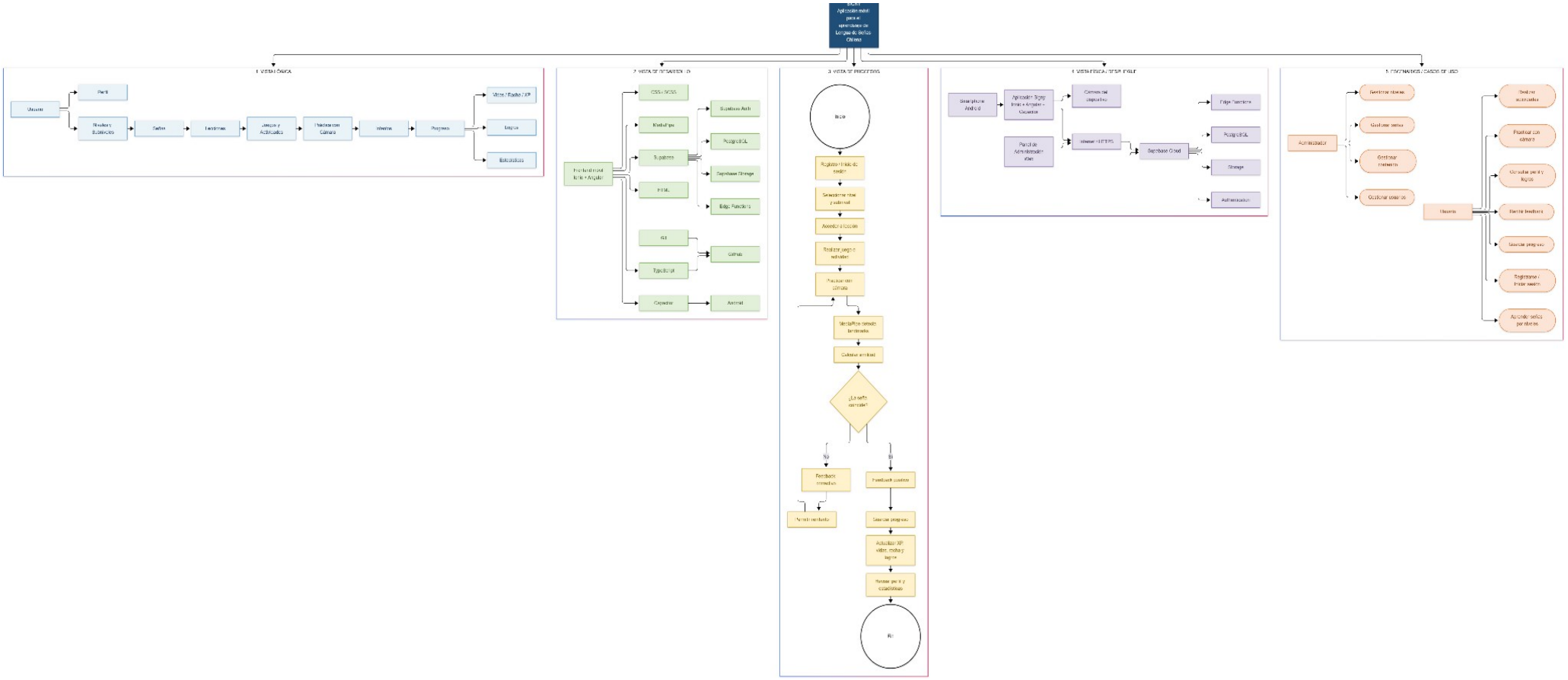


Figura 6. Vistas 4+1 de la arquitectura de Signy.
Fuente: elaboración propia del equipo Signy (2026), a partir de la arquitectura, tecnologías, procesos y escenarios definidos para la solución.

El modelo 4+1 permite comprobar que la arquitectura propuesta mantiene coherencia entre lo que el usuario necesita, los componentes que se desarrollan, los procesos que ejecuta la aplicación, la infraestructura donde opera y los escenarios que deben ser soportados. Para

Signy, esta vista es especialmente útil porque integra el frontend Ionic/Angular, Capacitor, MediaPipe, Supabase/PostgreSQL y la experiencia de aprendizaje dentro de una representación común.

Stack tecnológico

Para facilitar la lectura y justificar cada elección, el stack se presenta en formato numerado en lugar de una tabla:

1. Ionic: se utiliza para construir la interfaz móvil y reutilizar componentes web en una experiencia orientada a teléfonos. Se eligió porque se integra naturalmente con Angular y Capacitor.
2. Angular: estructura el frontend mediante componentes, servicios, rutas y formularios. Se eligió por su organización modular y por permitir separar presentación, lógica y acceso a datos.
3. TypeScript: lenguaje principal del frontend. Aporta tipado estático, facilita detectar errores antes de ejecución y mejora la mantenibilidad.
4. Capacitor: puente entre la aplicación web y capacidades nativas del dispositivo. Permite integrar cámara, notificaciones locales y compilación Android.
5. Supabase Auth: administra registro, inicio de sesión, sesiones, recuperación de contraseña y MFA/2FA, reduciendo la necesidad de implementar autenticación desde cero.
6. Supabase PostgreSQL: almacena usuarios extendidos, niveles, subniveles, señas, progreso, estadísticas, fallos, intentos y logros mediante un modelo relacional.
7. Supabase Storage: centraliza imágenes, GIF, WebP y videos utilizados por las lecciones y el panel de administración.
8. Supabase Edge Functions: ejecuta lógica serverless que debe correr fuera del cliente, como procesos de recuperación o eliminación de cuenta.
9. MediaPipe Tasks Vision / Hand Landmarker: detecta en tiempo real los 21 puntos (landmarks) de cada mano, que luego usa el clasificador propio para reconocer la seña.
10. Git y GitHub: proporcionan control de versiones, respaldo, colaboración y trazabilidad de cambios. Git gestiona el historial local y GitHub centraliza el repositorio del equipo.
11. TensorFlow / Keras (Python): entrena el clasificador propio con las muestras del dataset (ml/entrenar.py) y lo exporta a un archivo JSON que la app ejecuta sin conexión.
12. three.js: dibuja el avatar 3D del AI Studio, que muestra la seña de referencia mientras el administrador graba muestras.
13. Jasmine, Karma y unittest de Python: ejecutan las 664 pruebas unitarias de la app y las 69 del pipeline de IA.

Modelo de datos

El modelo de datos se apoya en PostgreSQL/Supabase y tiene 16 tablas propias más auth.users, todas con RLS. auth.users contiene la identidad de autenticación; profiles amplía los datos de usuario y follows guarda quién sigue a quién; user_stats mantiene vidas, racha, XP y congeladores, y racha_historial el calendario de práctica; niveles, subniveles y senas estructuran el contenido; progreso_nivel_usuario y progreso_subnivel_usuario almacenan el avance; practica_fallos e intentos_ejercicio registran errores e intentos; logros y usuario_logros soportan las insignias; regalos_congelador guarda los congeladores regalados entre amigos; password_reset_codes, los códigos de recuperación; y muestras_sena, el dataset de la IA (solo puntos de las manos, sin video). Las relaciones utilizan claves primarias/foráneas, restricciones de integridad y RLS; el XP y la racha se calculan en funciones de la base de datos. La Figura 7 muestra las tablas con las que trabaja la app; password_reset_codes no aparece porque solo la usa la Edge Function de recuperación de contraseña.

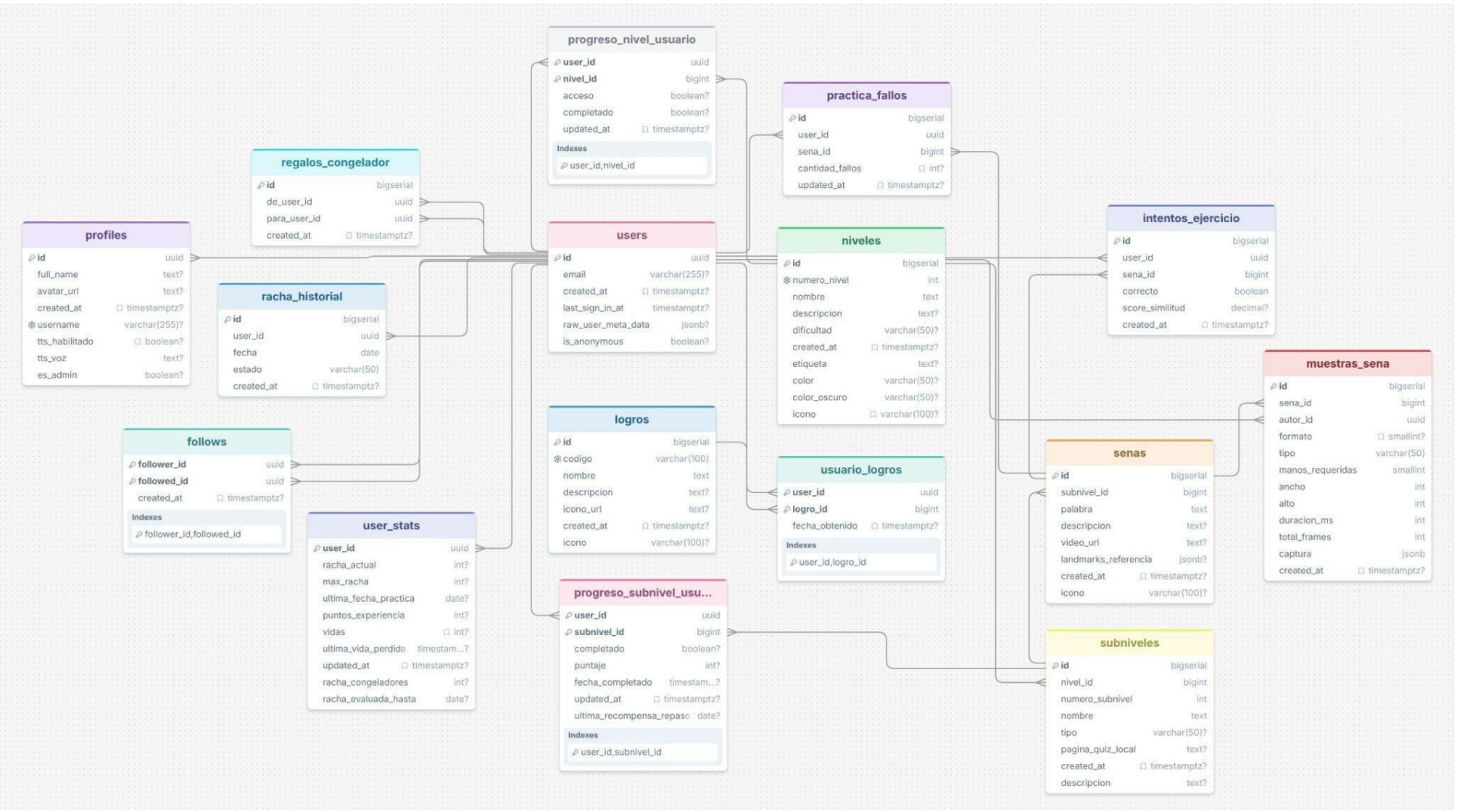


Figura 7. Modelo de datos base de Signy.

Diseño de interfaces / UX-UI

El flujo principal considera onboarding, registro o inicio de sesión, ruta de niveles, subnivel, lección, resultado y progreso, y perfil. Las lecciones se dividen en flashcards, emparejamiento, cuestionario/repaso y práctica con cámara. El diseño privilegia consistencia visual, retroalimentación inmediata, estados visibles, prevención de accesos a contenido no disponible y controles de tamaño adecuado. La accesibilidad se aborda mediante textos legibles, jerarquía clara, apoyo audiovisual y reducción de pasos innecesarios.

Diseño de seguridad

La seguridad se apoya en Supabase Auth, MFA/2FA, validación de sesiones, guards de navegación, roles administrativos y políticas Row Level Security. Los secretos no deben almacenarse en el repositorio y deben gestionarse mediante variables de entorno o configuración protegida. El acceso a cámara y notificaciones se ejecuta solo con permisos del dispositivo, y las operaciones sensibles se separan del cliente cuando corresponde mediante Edge Functions.

Integraciones

Las integraciones también se presentan numeradas para explicar qué intercambia cada componente, para qué se utiliza y qué ocurre ante errores:

1. Supabase Auth: intercambia credenciales, sesión e identificador de usuario. Su finalidad es autenticar y proteger la cuenta. Ante error se informa al usuario y se invalida o renueva la sesión según corresponda.
2. Supabase PostgreSQL: intercambia contenido, progreso, estadísticas, fallos e intentos. Es la persistencia principal. Los errores de consulta deben capturarse sin dejar la interfaz en un estado inconsistente.

3. Supabase Storage: intercambia archivos multimedia y sus rutas. Permite almacenar recursos de las señas. Si falla una descarga se informa el error y, cuando exista, se reutiliza la caché local.
4. Supabase Edge Functions: intercambia parámetros de procesos serverless, como recuperación o eliminación de cuenta. Las respuestas deben validarse y transformarse en mensajes comprensibles.
5. MediaPipe Hands: recibe fotogramas y devuelve coordenadas de landmarks, que el clasificador propio usa para reconocer la seña. Descarga su modelo de manos la primera vez que se usa la cámara; si el delegado GPU no está disponible, se usa la CPU. Si no detecta correctamente una mano, el usuario puede repetir el intento.
6. Cámara del dispositivo: entrega imagen en tiempo real para la práctica. Requiere permiso explícito; si no está disponible, la aplicación debe informar que la actividad no puede ejecutarse.
7. Capacitor Local Notifications: programa recordatorios y avisos de recuperación de vidas. Si el permiso está denegado, la función queda desactivada sin bloquear el resto de la app.
8. Servicio de texto a voz del dispositivo/navegador: reproduce vocabulario cuando el usuario lo activa. Si el servicio no está disponible, la lección debe continuar sin audio.
9. Capacitor / Android Runtime: conecta el código Ionic con APIs nativas y permite generar la versión Android utilizada en la validación académica.
10. GitHub: centraliza el repositorio del proyecto y el historial de cambios. Aunque no forma parte del flujo de usuario, funciona como integración de gestión técnica para trazabilidad, revisión y respaldo del desarrollo.

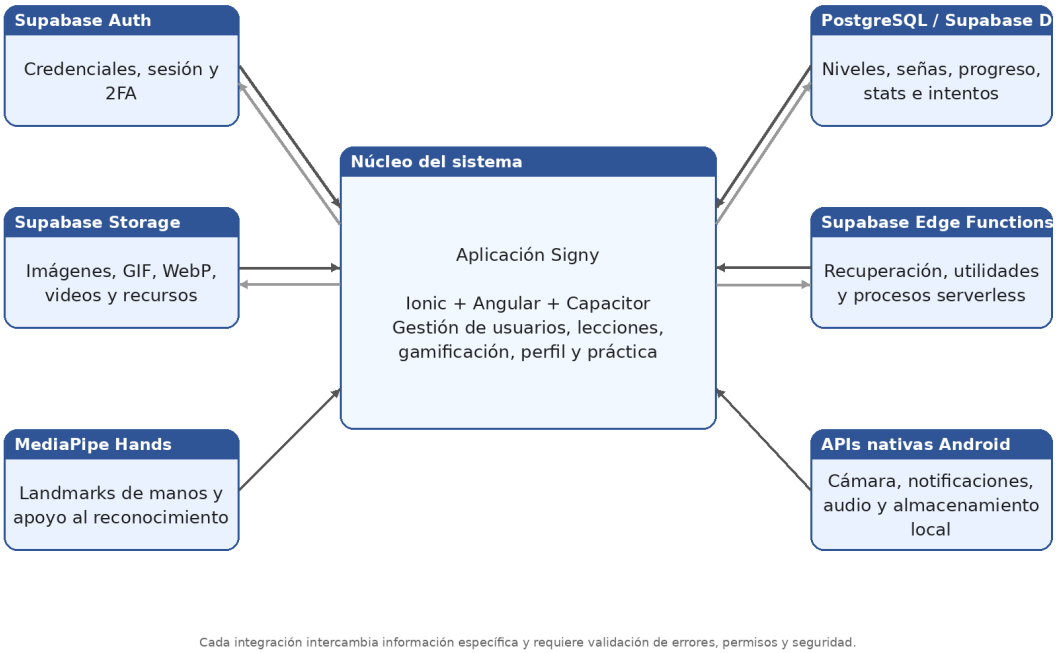


Figura 8. Diagrama de integraciones de Signy. Fuente: elaboración propia del equipo Signy (2026).

F. Diseño específico según naturaleza del proyecto

Componente IA / Visión Computacional

1. Origen de los datos: la cámara del dispositivo entrega fotogramas durante la práctica. MediaPipe Hands (Zhang et al., 2020) procesa cada fotograma nuevo y detecta 21 landmarks por mano, con GPU y respaldo en CPU.

2. Dataset: Signy no entrena un detector de manos desde cero; usa el modelo preentrenado de MediaPipe. Sobre esa base, el equipo creó un dataset propio (tabla muestras_sena, con RLS solo para administradores) que guarda únicamente coordenadas de las manos, sin imágenes ni video, grabadas en el AI Studio del panel de administración junto con el apodo del participante. El modelo actual se entrenó con 169 muestras de las 12 señas.

3. Preparación de datos: las coordenadas se corrigen según la relación de aspecto de la cámara, se centran en la muñeca, se escalan según el tamaño de la mano y se remuestrean a un número fijo de fotogramas. Para ganar robustez se aplica aumentación: espejo (personas zurdas), giro, escala, temblor y deformación temporal.

4. Modelo: un clasificador propio, una red neuronal MLP de Keras/TensorFlow (Abadi et al., 2016) con dos capas ocultas (64 y 32 neuronas) y una salida softmax con una clase extra para «ninguna seña», recibe 12 fotogramas normalizados (1.524 valores). Se entrena en Python (ml/entrenar.py) y se exporta a un archivo JSON que la app ejecuta sin conexión; una prueba de paridad verifica que Python y la app calculen las mismas características. En la lección la seña se aprueba cuando la IA le asigna al menos 75 % de seguridad (el umbral calibrado en el entrenamiento, cercano a 95 %, rechazaba señas bien hechas). La retroalimentación depende del resultado: entre 75 y 89 % se aprueba con hasta dos consejos reales para mejorar, entre 90 y 99 % con un consejo opcional, y al 100 % solo se felicita, con un trofeo y confeti. Las señas que en LSCh se hacen igual, como «Hola» y «Chao», se aceptan como equivalentes. La comparación con grabaciones de referencia mediante Dynamic Time Warping (Sakoe y Chiba, 1978), usada hasta octubre, se eliminó el 25/10 porque el clasificador reconocía mejor y DTW hacía más lento el celular. MediaPipe por sí solo no decide si la seña es correcta.

5. Inferencia:

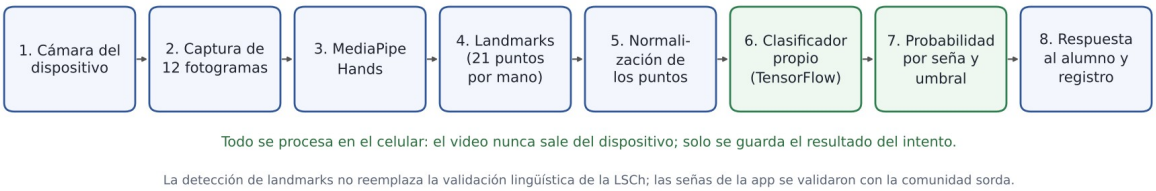


Figura 9. Flujo del componente de IA / visión computacional. Fuente: elaboración propia del equipo Signy (2026).

Métricas del componente

1. Probabilidad que asigna el clasificador a la seña pedida.
2. Porcentaje de intentos reconocidos correctamente.
3. Tasa de falsos positivos.
4. Tasa de falsos negativos.
5. Latencia de procesamiento.
6. Fotogramas procesados por segundo (FPS).
7. Estabilidad de detección de landmarks.

Estado actual de las métricas. Con el conjunto de prueba (44 muestras separadas del entrenamiento), el clasificador acierta el 70,5 % de las señas y el 90,3 % cuando supera el umbral de

confianza, y reconoce el 72,7 % de los casos en que no se hizo ninguna seña. Al evaluarlo con personas que no participaron en el entrenamiento, la exactitud baja a 27,8 %: el modelo necesita muestras de más personas. En rendimiento, la app usa un solo detector compartido, procesa como máximo unos 22 cuadros por segundo y pasa de GPU a CPU si es necesario. Estas cifras provienen del reporte de entrenamiento guardado junto al modelo (26/10/2026).

Validación

La validación se divide en dos niveles. La validación técnica comprueba que la cámara, MediaPipe y el clasificador funcionen de forma estable; la validación lingüística comprueba que la seña enseñada y aceptada corresponda realmente a LSCh. Esta segunda validación se realizó con personas de la comunidad sorda, coordinadas por DIDECO Huechuraba: las señas de la app se grabaron con ellas, porque una coincidencia geométrica no garantiza corrección lingüística.

Limitaciones y justificación

1. Iluminación: la detección visual necesita contraste suficiente; si la escena es oscura o tiene reflejos, la calidad de los landmarks disminuye.
2. Oclusiones: cuando los dedos o una mano tapan a la otra, el modelo puede perder precisión y generar landmarks incompletos.
3. Variabilidad entre usuarios: el tamaño de las manos, la velocidad del movimiento y la forma de ejecutar una seña cambian entre personas; con el dataset actual la exactitud con personas nuevas es baja, por lo que se necesitan muestras de más personas.
4. Expresiones no manuales: muchas señas dependen también de orientación corporal y expresiones faciales; un análisis basado solo en manos no captura toda la dimensión lingüística.
5. Validación lingüística acotada: las 12 señas actuales se validaron con la comunidad sorda, pero cada seña nueva necesitará la misma revisión antes de publicarse.
6. Alcance Android en Fase 2: se decidió validar primero en Android porque Ionic + Capacitor y el acceso a cámara/notificaciones ya están siendo trabajados en ese entorno. Extender a iOS implica más pruebas, permisos y tiempo académico, por lo que queda fuera del alcance inmediato.

Componentes especializados no aplicables

1. Georreferenciación: no aplica porque el valor principal de Signy no depende de ubicación, mapas, GPS ni reglas espaciales. Agregarlo aumentaría alcance sin resolver el problema central del aprendizaje de LSCh.
2. IoT/Hardware especializado: no aplica porque el proyecto no requiere sensores externos, microcontroladores ni dispositivos dedicados; utiliza hardware estándar del teléfono, principalmente cámara y capacidades de notificación.
3. Motor experto de reglas: no es el componente principal. Existen reglas de negocio para vidas, rachas, acceso y logros, pero no se necesita un motor de inferencia separado; estas reglas pueden resolverse en la lógica de aplicación y base de datos.
4. iOS como plataforma objetivo en esta fase: se excluye por alcance y factibilidad. El framework es multiplataforma, pero la validación académica se concentra en Android porque es el entorno disponible para compilación/pruebas del equipo. iOS requeriría infraestructura y pruebas adicionales específicas, por lo que se considera una posible extensión futura.

G. Planificación Ágil

Metodología de trabajo

Signy se gestiona mediante Scrum con Product Backlog priorizado, Sprints, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective y refinamiento continuo, según la Guía de Scrum (Schwaber y Sutherland, 2020). El tablero del equipo se gestiona en Trello y la trazabilidad técnica en GitHub. El objetivo es entregar valor de forma incremental, controlar cambios de alcance y hacer visible el trabajo pendiente, en curso y terminado.

Squad y responsabilidades

Integrante	Rol	Responsabilidades principales
Fernanda Poblete	Product Owner	Mantener visión, priorizar backlog, definir criterios de aceptación, controlar alcance y validar incrementos.
Catalina Gutiérrez	Scrum Master / Development Team	Facilitar Scrum, registrar impedimentos, coordinar ceremonias y aportar técnicamente.
Benjamín Vallejos	Development Team	Desarrollo e integración de funcionalidades frontend, seguridad, administración, caché, notificaciones y gamificación.
Michael Ibáñez	Development Team	Desarrollo, versionado, corrección de errores, integración y estabilización.

Épicas

1. Autenticación y seguridad de cuenta.
2. Contenido LSCh y estructura de aprendizaje.
3. Flujo de aprendizaje.
4. Gamificación y progreso.
5. Perfil y comunidad.
6. Administración de contenido.
7. Reconocimiento de señas mediante visión computacional.
8. Validación lingüística.
9. Hábito y experiencia inicial.
10. Rendimiento y optimización.

Product Backlog priorizado

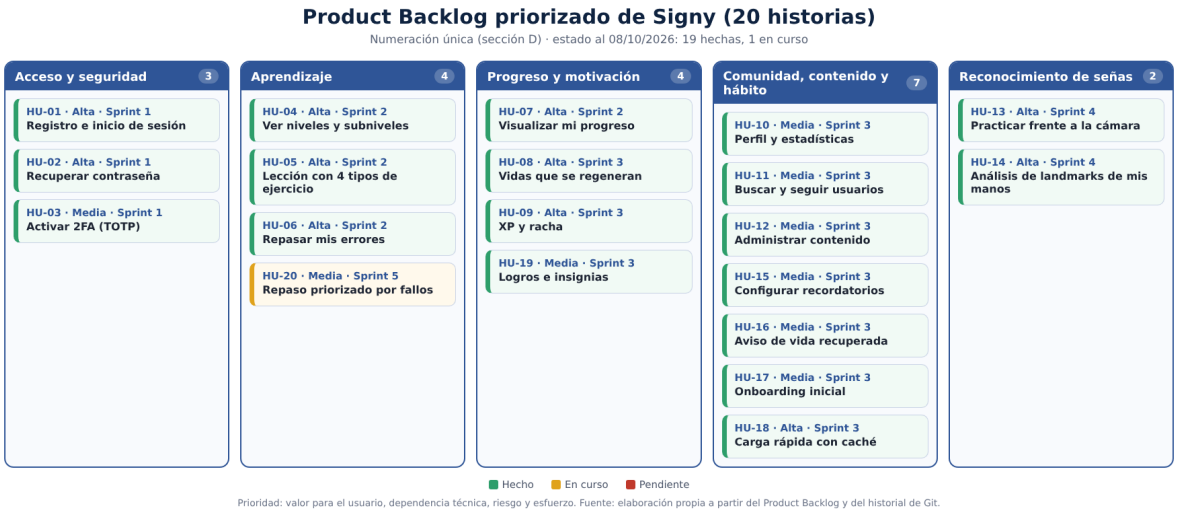


Figura 10. Product Backlog priorizado de Signy (20 historias). Fuente: elaboración propia del equipo Signy (2026).

Para simplificar la planificación, el Product Backlog se divide en dos bloques de 10 historias. El Bloque 1 (HU-01 a HU-10) concentra el núcleo funcional y la base ya consolidada; el Bloque 2 (HU-11 a HU-20) reúne reconocimiento, validación y mejoras de aprendizaje. Esta división no significa que solo existan dos Sprints: los 20 elementos se distribuyen en cinco Sprints para mantener cargas de trabajo manejables y permitir revisiones frecuentes. La numeración de las historias es única en todo el informe y corresponde a la tabla de la sección D.

Criterio de priorización

La prioridad considera valor para el usuario, dependencia técnica, riesgo, urgencia y esfuerzo. Primero se asegura el flujo mínimo utilizable; luego se incorporan gamificación y comunidad; finalmente se abordan los elementos con mayor incertidumbre, como reconocimiento con MediaPipe, validación lingüística y personalización del repaso.

User Story Mapping

El User Story Mapping (Patton, 2014) ordena las historias según el recorrido del usuario: acceder, aprender, practicar, progresar, interactuar y administrar.

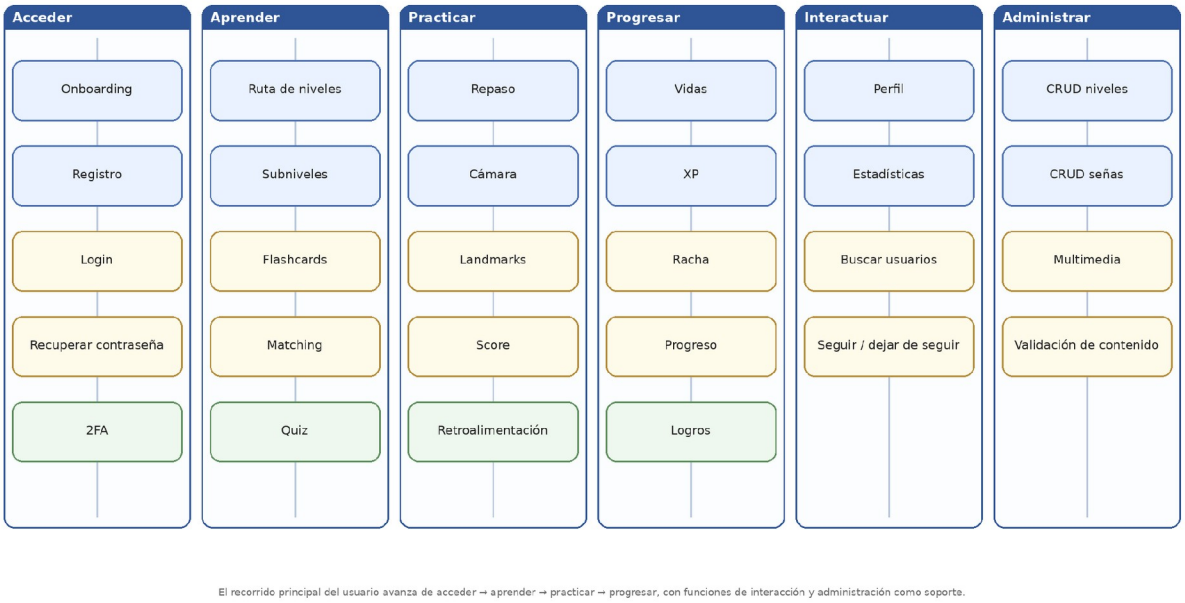


Figura 11. User Story Mapping de Signy. Fuente: elaboración propia del equipo Signy (2026).

Release Planning

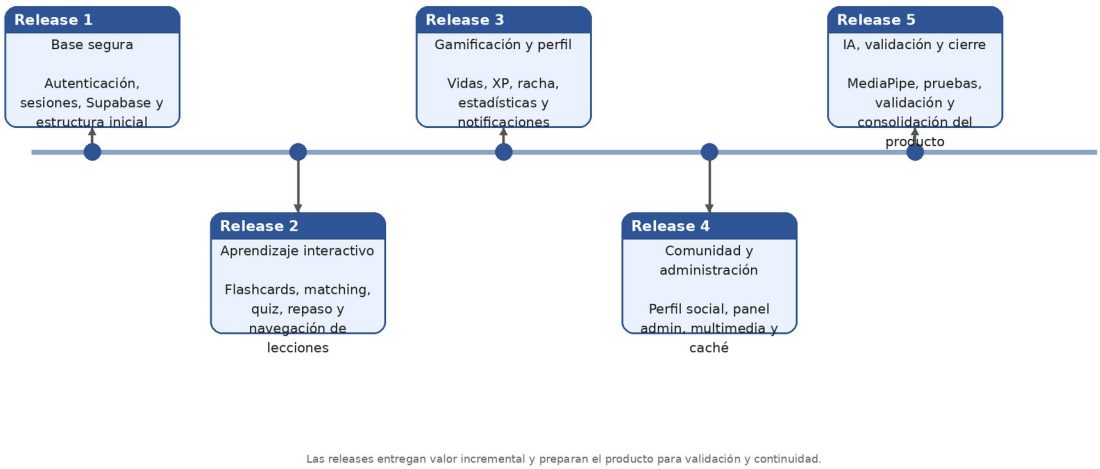


Figura 12. Release Planning de Signy (5 releases). Fuente: elaboración propia del equipo Signy (2026).

H. Ejecución iterativa — Sprints

La planificación se organiza en cinco Sprints. El antiguo Sprint 4 se dividió en dos: uno dedicado a integrar MediaPipe y otro a validar, corregir y cerrar el producto (CC-003). Como el plan de cinco Sprints se formalizó el 10/09/2026, los Sprints 1 a 3 agrupan el trabajo realizado entre el 13/08 y el 08/09; el Sprint 4 se ejecutó entre el 09/09 y el 04/10, y el Sprint 5 desde el 05/10 hasta el cierre. El release burndown muestra el avance real del backlog, reconstruido a partir de la fecha del commit que dejó funcional cada historia: la última (HU-20) se cerró el 26/10/2026.

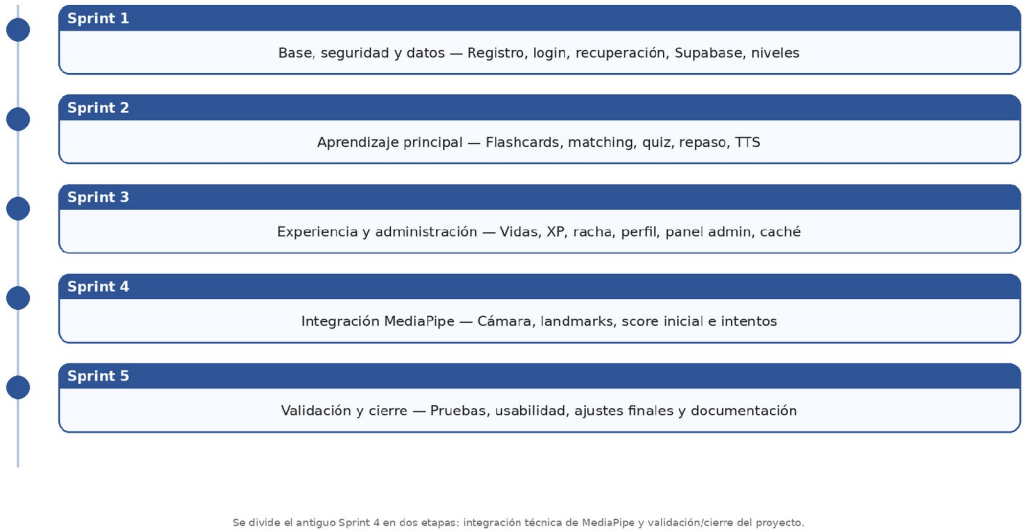


Figura 13. Roadmap de Sprints propuesto para Signy. Fuente: elaboración propia del equipo Signy (2026).

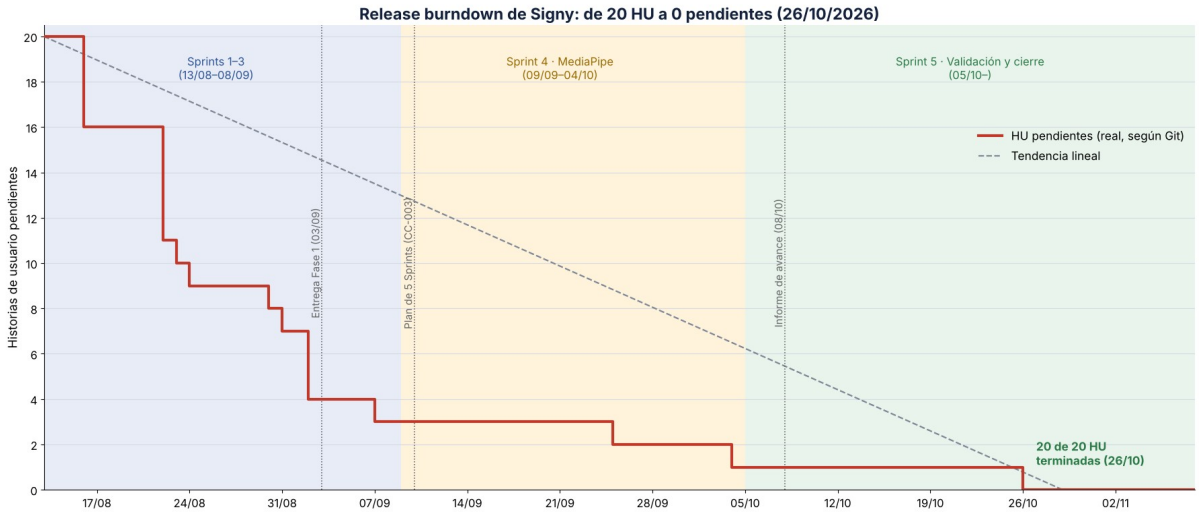


Figura 14. Release burndown de Signy (20 historias de usuario).
Fuente: elaboración propia del equipo Signy (2026), a partir del historial de commits del repositorio.

Sprint 1 – Base, seguridad y datos

Sprint Goal: Consolidar el acceso seguro y la persistencia real del producto.

Sprint Backlog: HU-01, HU-02 y HU-03, más la base técnica de Supabase (T-01). Estado: completado.

Scrum Board:

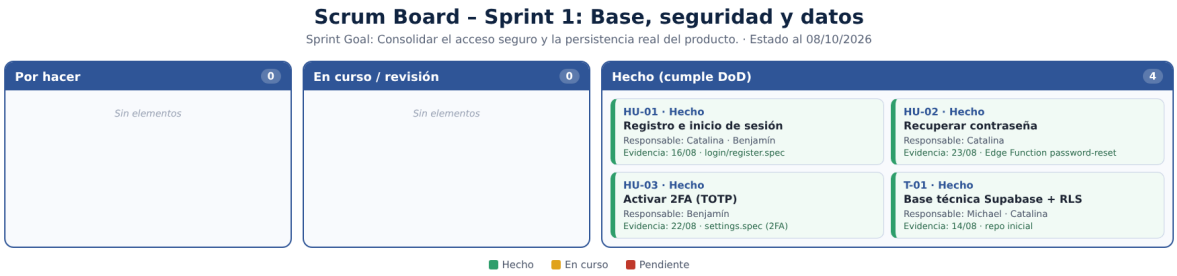


Figura 15. Scrum Board – Sprint 1: Base, seguridad y datos.

Fuente: elaboración propia del equipo Signy (2026), a partir del Product Backlog y del historial del repositorio; estado al 08/11/2026.

Burndown Chart: las tres historias del Sprint quedaron terminadas entre el 16/08 y el 23/08 (ver el release burndown).

Daily Meetings: reuniones breves para revisar avance, siguiente acción e impedimentos. Los bloqueos se registran en el Impediment Log.

Impediment Log: Configuración de Supabase, autenticación, sesiones, recuperación de contraseña y migración desde almacenamiento local.

Definition of Done: criterios de aceptación cumplidos, integración estable, compilación sin errores críticos, prueba funcional ejecutada, documentación necesaria actualizada y evidencia disponible para la Review.

Sprint Review: el incremento se demuestra al equipo y al docente; el Product Owner acepta, rechaza o devuelve historias al backlog según criterios de aceptación.

Retrospective: el equipo identifica qué mantener, qué detener y qué mejorar para el Sprint siguiente, dejando acciones concretas y responsables.

Sprint 2 – Aprendizaje principal

Sprint Goal: Completar el flujo de aprendizaje y registrar avance.

Sprint Backlog: HU-04, HU-05, HU-06 y HU-07, más el texto a voz en la lección (T-02). Estado: completado.

Scrum Board:

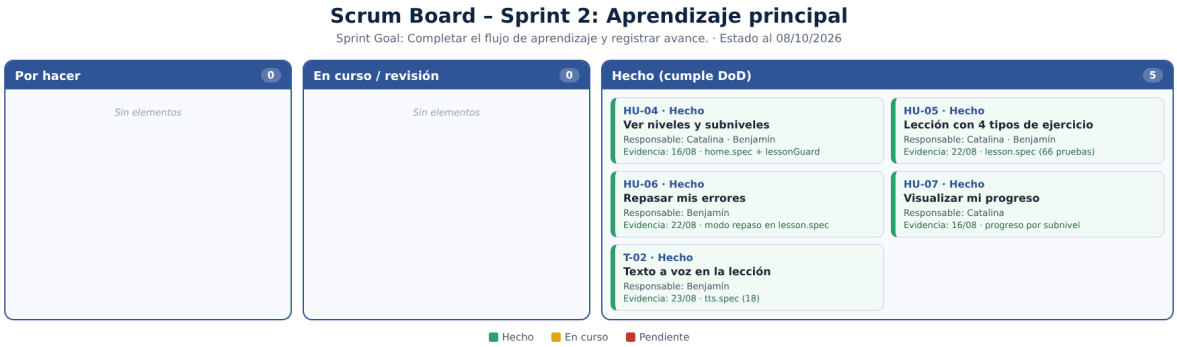


Figura 16. Scrum Board – Sprint 2: Aprendizaje principal.

Fuente: elaboración propia del equipo Signy (2026), a partir del Product Backlog y del historial del repositorio; estado al 08/11/2026.

Burndown Chart: las cuatro historias del Sprint quedaron terminadas entre el 16/08 y el 22/08; la lección se estabilizó el 30/08 con el emparejamiento con señas reales.

Daily Meetings: reuniones breves para revisar avance, siguiente acción e impedimentos. Los bloqueos se registran en el Impediment Log.

Impediment Log: Coordinación entre fases, persistencia del progreso, lógica de vidas y consistencia del estado del usuario.

Definition of Done: criterios de aceptación cumplidos, integración estable, compilación sin errores críticos, prueba funcional ejecutada, documentación necesaria actualizada y evidencia disponible para la Review.

Sprint Review: el incremento se demuestra al equipo y al docente; el Product Owner acepta, rechaza o devuelve historias al backlog según criterios de aceptación.

Retrospective: el equipo identifica qué mantener, qué detener y qué mejorar para el Sprint siguiente, dejando acciones concretas y responsables.

Sprint 3 – Experiencia, comunidad y administración

Sprint Goal: Fortalecer la experiencia del usuario y facilitar la gestión del contenido.

Sprint Backlog: HU-08 a HU-12 y HU-15 a HU-19, más el endurecimiento de seguridad (T-03). HU-19 se adelantó desde el Sprint 5. Estado: completado.

Scrum Board:



Figura 17. Scrum Board – Sprint 3: Experiencia, comunidad y administración.

Fuente: elaboración propia del equipo Signy (2026), a partir del Product Backlog y del historial del repositorio; estado al 08/11/2026.

Burndown Chart: las diez historias del Sprint se cerraron entre el 16/08 y el 07/09; varias se adelantaron porque compartían componentes con los Sprints 1 y 2.

Daily Meetings: reuniones breves para revisar avance, siguiente acción e impedimentos. Los bloqueos se registran en el Impediment Log.

Impediment Log: Compatibilidad Android, permisos, multimedia, caché, notificaciones y sincronización de datos sociales.

Definition of Done: criterios de aceptación cumplidos, integración estable, compilación sin errores críticos, prueba funcional ejecutada, documentación necesaria actualizada y evidencia disponible para la Review.

Sprint Review: el incremento se demuestra al equipo y al docente; el Product Owner acepta, rechaza o devuelve historias al backlog según criterios de aceptación.

Retrospective: el equipo identifica qué mantener, qué detener y qué mejorar para el Sprint siguiente, dejando acciones concretas y responsables.

Sprint 4 – Integración MediaPipe

Sprint Goal: Conectar la detección de landmarks con la práctica real de la lección y registrar intentos.

Sprint Backlog: HU-13 y HU-14, más el registro de intentos con su score (T-04) y el entrenamiento del clasificador (T-05). Estado: completado; T-05 se terminó en el Sprint 5, el 26/10, con el modelo propio entrenado con TensorFlow.

Scrum Board:

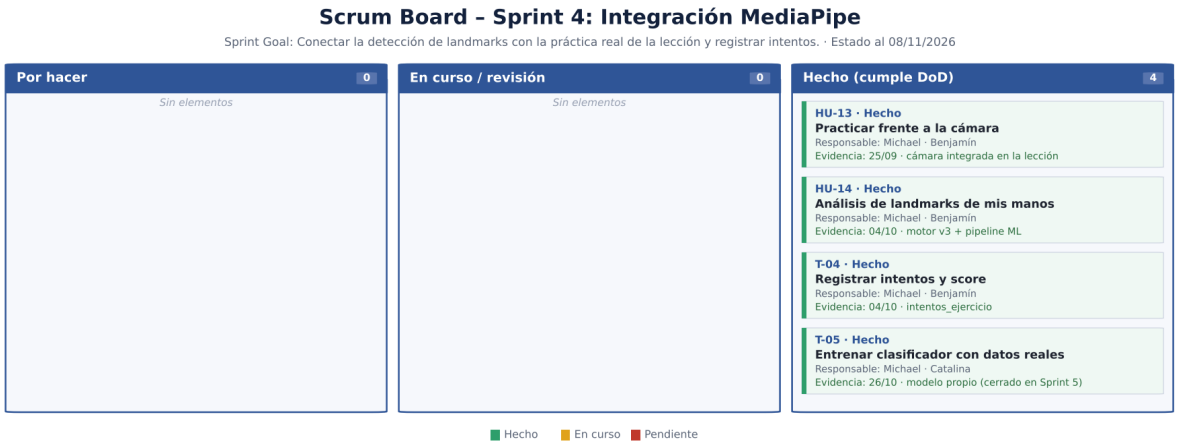


Figura 18. Scrum Board – Sprint 4: Integración MediaPipe.

Fuente: elaboración propia del equipo Signy (2026), a partir del Product Backlog y del historial del repositorio; estado al 08/11/2026.

Burndown Chart: HU-13 se cerró el 25/09 (cámara integrada en la lección) y HU-14 el 04/10 (motor v3), con lo que quedó una sola historia pendiente en el backlog.

Daily Meetings: reuniones breves para revisar avance, siguiente acción e impedimentos. Los bloqueos se registran en el Impediment Log.

Impediment Log: Rendimiento en dispositivo, iluminación, calibración del score, variación entre usuarios y estabilidad de cámara.

Definition of Done: criterios de aceptación cumplidos, integración estable, compilación sin errores críticos, prueba funcional ejecutada, documentación necesaria actualizada y evidencia disponible para la Review.

Sprint Review: el incremento se demuestra al equipo y al docente; el Product Owner acepta, rechaza o devuelve historias al backlog según criterios de aceptación. Resultado: la práctica con cámara reconoce señas reales con el motor v3 y cada intento queda registrado con su score.

Retrospective: el equipo identifica qué mantener, qué detener y qué mejorar para el Sprint siguiente, dejando acciones concretas y responsables.

Sprint 5 – Validación, personalización y cierre

Sprint Goal: Validar la solución, incorporar mejoras de aprendizaje y preparar el cierre de la fase.

Sprint Backlog: HU-20; pruebas automatizadas (T-06), pruebas manuales en Android (T-07), pruebas de usabilidad (T-08) y validación lingüística (T-09), además de AI Studio con avatar 3D (T-10) y Dashboard de KPIs (T-11). Estado: completado.

Scrum Board:

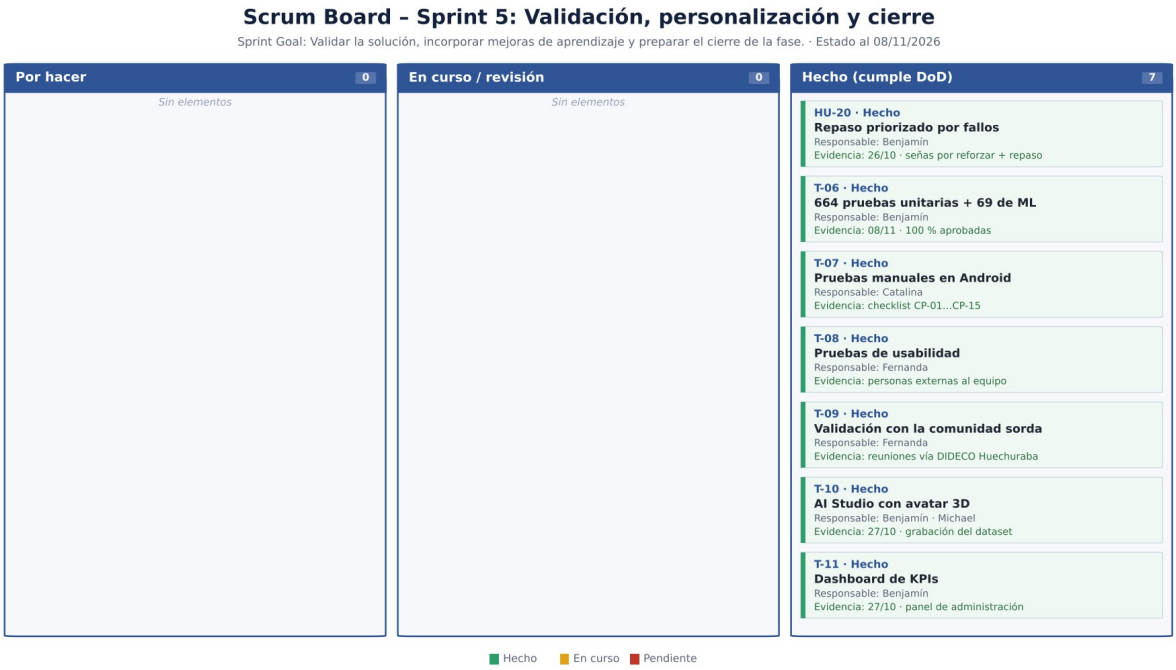


Figura 19. Scrum Board – Sprint 5: Validación, personalización y cierre.

Fuente: elaboración propia del equipo Signy (2026), a partir del Product Backlog y del historial del repositorio; estado al 08/11/2026.

Burndown Chart: HU-20 se cerró el 26/10, con lo que las 20 historias quedaron terminadas; las tareas de pruebas y validación también se completaron dentro del Sprint.

Daily Meetings: reuniones breves para revisar avance, siguiente acción e impedimentos. Los bloqueos se registran en el Impediment Log.

Impediment Log: Disponibilidad de validadores externos, tiempos de prueba, ajustes derivados del feedback y cierre de defectos prioritarios.

Definition of Done: criterios de aceptación cumplidos, integración estable, compilación sin errores críticos, prueba funcional ejecutada, documentación necesaria actualizada y evidencia disponible para la Review.

Sprint Review: el incremento se demuestra al equipo y al docente; el Product Owner acepta, rechaza o devuelve historias al backlog según criterios de aceptación.

Retrospective: el equipo identifica qué mantener, qué detener y qué mejorar para el Sprint siguiente, dejando acciones concretas y responsables.

Seguimiento y control de la gestión ágil

El Product Owner mantiene el backlog, prioridad, alcance y criterios de aceptación; el Scrum Master facilita el marco de trabajo y gestiona impedimentos; el Development Team actualiza tareas, integra cambios y reporta desviaciones. La trazabilidad se mantiene entre historias, tareas, commits, pruebas y evidencias. Los cambios de alcance se evalúan primero en el Product Backlog antes de comprometerse en un Sprint.

I. Estrategia y ejecución de pruebas

Plan de pruebas

El plan de pruebas busca verificar que Signy cumpla los requisitos funcionales y no funcionales definidos, con énfasis en autenticación, persistencia, flujo de lecciones, gamificación, administración, cámara, MediaPipe, rendimiento y seguridad. La estrategia combina pruebas manuales funcionales, pruebas de integración entre frontend y Supabase, revisión de permisos y sesiones, pruebas de usabilidad y mediciones técnicas del componente visual.

Ambientes: desarrollo local/web para depuración rápida; Android en emulador o dispositivo para cámara, notificaciones y comportamiento móvil; Supabase como backend compartido de desarrollo/pruebas; y una configuración de demostración estable para las revisiones del proyecto.

Criterios de entrada: versión compilable, historias seleccionadas con criterios de aceptación definidos, datos de prueba disponibles y servicios externos operativos.

Criterios de salida: casos críticos aprobados, sin defectos bloqueantes/altos abiertos, defectos medios documentados, evidencias guardadas y Product Owner informado del resultado.

Casos de prueba funcionales

ID	Req.	Caso	Resultado esperado	Resultado obtenido	Estado
CP-01	RF-01/RF-02	Registro e inicio de sesión válidos	Cuenta creada y sesión iniciada correctamente	Pruebas de login, registro, validadores y guards aprobadas; verificado también en Android.	Aprobado *
CP-02	RF-03	Recuperación de contraseña	Código enviado y cambio de contraseña realizado	Navegación de la pantalla verificada en pruebas unitarias; envío real del código probado manualmente en Android.	Aprobado
CP-03	RF-04	Activación 2FA	Segundo factor configurado y solicitado cuando corresponda	Activación y desactivación de 2FA verificadas en SettingsPage.	Aprobado *
CP-04	RF-05/RF-06	Visualización y bloqueo de niveles	Contenido disponible visible y subniveles vacíos bloqueados	Bloqueo de subniveles sin contenido verificado en lessonGuard y HomePage.	Aprobado *
CP-05	RF-08/RF-09	Lección de cuatro fases y repaso	Usuario completa flujo y errores pasan a repaso	Cuatro fases, señas «señuelo» y modo repaso verificados en LessonPage.	Aprobado *
CP-06	RF-11	Persistencia de progreso	Avance permanece tras cerrar y volver a iniciar sesión	Persistencia verificada con cliente Supabase simulado (ContenidoService).	Aprobado *
CP-07	RF-12	Regeneración de vidas	Vida descontada y regenerada según regla temporal	Descuento y regeneración de vidas verificados en ContenidoService.	Aprobado *
CP-08	RF-13/RF-14	Racha y protección	Racha se actualiza y aplica la regla configurada	Racha, congeladores y calendario verificados en ContenidoService y RachaCalendar.	Aprobado *
CP-09	RF-17/RF-18	CRUD administrativo	Administrador crea/edita/elimina contenido y multimedia	CRUD y reglas de duplicados verificados en AdminVocabularioPage.	Aprobado *

ID	Req.	Caso	Resultado esperado	Resultado obtenido	Estado
CP-10	RF-19/RF-20	Cámara y landmarks	Cámara inicia y MediaPipe detecta landmarks de mano	MediaPipe, clasificador y prueba de paridad Python/app verificados; cámara real probada en Android.	Aprobado *
CP-11	RF-21/RF-28	Feedback y registro de intento	Resultado visible y registro persistido cuando corresponda	Aprobación desde 75 %, retroalimentación por niveles (felicitación al 100 %) y registro del intento con su score verificados.	Aprobado *
CP-12	RF-22/RF-23	Caché multimedia	Segunda apertura reutiliza recurso local y muestra carga coherente	Caché de medios verificada en ImageCacheService y en Android.	Aprobado *
CP-13	RF-24/RF-25	Notificaciones locales	Recordatorio/vida recuperada se programa con permisos activos	Programación de recordatorios y aviso de vida verificados; recibidos en Android.	Aprobado *
CP-14	RF-29	Logros	Se desbloquea logro al cumplir condición y no se duplica	Otorgamiento de logros sin duplicados verificado en ContenidoService.	Aprobado *
CP-15	RF-30	Repaso inteligente	Se priorizan señas con más fallos	Registro de fallos y lista «señas por reforzar» ordenada por fallos (getMisFallos) verificados.	Aprobado *

** Aprobado mediante pruebas unitarias automatizadas con dependencias simuladas, ejecutadas el 08/11/2026 (664 aprobadas, 1 omitida, 0 fallidas). Los flujos principales se verificaron además manualmente en un dispositivo Android físico (T-07). Las capturas de la ejecución están en el Anexo de evidencias (Figuras 22 a 24).*

Pruebas no funcionales

ID	Tipo	Prueba	Criterio esperado
PNF-01	Seguridad	Intentar acceder a datos de otro usuario	RLS impide lectura/escritura no autorizada.
PNF-02	Rendimiento	Medir carga de lección con y sin caché	La segunda carga no debe volver a descargar multimedia ya almacenada.
PNF-03	Usabilidad	Prueba guiada con usuarios	El usuario completa tareas principales sin asistencia crítica.
PNF-04	Compatibilidad Android	Ejecutar flujo en dispositivo/emulador	No deben existir bloqueos críticos en navegación, cámara ni notificaciones.
PNF-05	IA/latencia	Medir FPS y latencia de MediaPipe	Retroalimentación fluida: un solo detector, máximo ~22 cuadros por segundo y respaldo de GPU a CPU.

ID	Tipo	Prueba	Criterio esperado
PNF-06	Accesibilidad	Revisar legibilidad y controles	Textos y controles comprensibles, sin elementos esenciales inaccesibles.
PNF-07	Mantenibilidad	Revisión de estructura y duplicación	Servicios/componentes con responsabilidades claras y sin duplicación crítica.

Gestión de defectos

Los defectos se registran con su severidad, corrección, re-prueba y estado. La tabla resume los defectos reales detectados durante la Fase 2 a partir del historial del repositorio:

ID	Defecto	Severidad	Corrección	Re-prueba	Estado
DF-01	Un usuario podía autoasignarse privilegios de administrador con un UPDATE directo a profiles.	Bloqueante	Script SQL proteger_es_admin (07/09).	Guard adminGuard y revisión de RLS	Cerrado
DF-02	El XP y la finalización de lecciones se aceptaban sin validar desde el cliente.	Alta	Lógica trasladada a PostgreSQL (19/09).	Pruebas de ContenidoService	Cerrado
DF-03	Diseño roto y botones sin respuesta en la app instalada (Android 16, edge-to-edge).	Alta	Ajustes de layout (27/08).	Prueba en dispositivo	Cerrado
DF-04	El motor de señas repetía fotogramas y una seña equivocada podía puntuar más que la correcta.	Alta	Motor v3 (04/10).	Pruebas de regresión del motor	Cerrado
DF-05	El botón atrás de Android cerraba la aplicación.	Media	Servicio boton-atras (04/10).	27 pruebas	Cerrado
DF-06	El historial de racha nunca se guardaba por falta de una restricción única.	Media	Script SQL fix_racha_historial_unique (07/09).	Pruebas de racha	Cerrado
DF-07	ng test no compilaba (tsconfig.spec incompleto y PracticaPage con métodos inexistentes).	Media	Configuración corregida y PracticaPage en cuarentena (10/09).	Suite completa	Cerrado
DF-08	La voz elegida en Configuración no sonaba en Android y había bajo contraste en selectores.	Baja	Ajustes del 04/10.	TtsService (18 pruebas)	Cerrado
DF-09	PracticaPage llama a métodos inexistentes y no está enrutada.	Media	Página fuera de uso: no está enrutada y sus pruebas siguen en cuarentena.	Sin impacto en la app	Abierto (baja prioridad)
DF-10	El modelo TFLite de ml/output espera 64 entradas y no trae su script de características.	Media	Se reemplazó por el modelo propio entrenado con TensorFlow y exportado a JSON (26/10).	Prueba de paridad Python/app	Cerrado

ID	Defecto	Severidad	Corrección	Re-prueba	Estado
DF-11	Con la cámara y la IA trabajando juntas el celular se ponía lento.	Alta	Un solo detector compartido, máximo ~22 cuadros por segundo, respaldo de GPU a CPU y eliminación de DTW (25/10).	Pruebas del motor y en Android	Cerrado
DF-12	Hola y Chao se rechazaban entre sí porque en LSCh se hacen con la misma seña.	Media	Se aceptan como señas equivalentes al evaluar (08/11).	3 pruebas nuevas	Cerrado
DF-13	El umbral calibrado (cercano a 95 %) rechazaba señas bien hechas.	Media	Aprobación desde 75 % y retroalimentación por niveles; al 100 %, felicitación (08/11).	7 pruebas nuevas	Cerrado
DF-14	La precarga de imágenes de la lección no tenía tiempo máximo: un servidor lento podía dejar pegada la barra de carga.	Baja	Tiempo máximo de 1,8 s, igual que en los videos (08/11).	1 prueba nueva	Cerrado

J. Validación de la solución

Validación con usuarios y referentes

La validación de Signy separó tres perspectivas: validación académica/técnica con el equipo y la docente; validación de usabilidad con personas oyentes externas al equipo; y validación lingüística/cultural con personas de la comunidad sorda, coordinada por la Oficina de Inclusión de DIDECO Huechuraba. Las tres se realizaron: la app funcionó correctamente con las personas que la probaron y las señas se grabaron con la comunidad sorda. El mayor obstáculo fue la demora en la comunicación para agendar las reuniones. Las actividades fueron:

1. Demostración técnica al docente y equipo utilizando una versión estable del Sprint.
2. Prueba de tareas con usuarios: registrarse, iniciar una lección, completar ejercicios, revisar progreso y configurar recordatorios.
3. Encuesta breve posterior a la prueba sobre facilidad de uso, claridad, motivación y problemas encontrados.
4. Registro de observaciones y defectos en el backlog con prioridad definida por el Product Owner.
5. Revisión y grabación de las señas con personas de la comunidad sorda antes de publicarlas en la app.

Cumplimiento de entregables

Entregable	Estado	Evidencia	Resultado
1. Autenticación y seguridad	Implementado	Registro, login, recuperación y 2FA; pruebas unitarias y prueba manual del correo	Cumple
2. Capa de datos Supabase	Implementado	16 tablas con RLS, funciones en PostgreSQL y 2 Edge Functions	Cumple
3. Lección de 4 fases	Implementado	Flashcards, emparejamiento, quiz, repaso y cámara con reconocimiento	Cumple
4. Gamificación	Implementado	Vidas, racha, XP, congeladores, logros y ranking	Cumple
5. Perfil y sistema social	Implementado	Perfil, estadísticas, búsqueda, seguidores y regalo de congeladores	Cumple
6. Panel administración	Implementado	CRUD, multimedia, AI Studio y Dashboard de KPIs	Cumple
7. Reconocimiento MediaPipe	Implementado	Cámara en la lección (25/09), motor v3 (04/10) y clasificador propio con TensorFlow (26/10)	Cumple
8. Validación lingüística	Realizado	Reuniones con personas de la comunidad sorda vía DIDECO Huechuraba; señas grabadas con ellas	Cumple; el mayor desafío fue la demora en la comunicación
9. Notificaciones	Implementado	Recordatorios y aviso de vida recuperada	Cumple

Entregable	Estado	Evidencia	Resultado
10. Caché multimedia	Implementado	Caché y precarga	Cumple
11. Onboarding/control contenido	Implementado	Onboarding y estado “próximamente”	Cumple

K. Evaluación económica y viabilidad

La evaluación económica se construyó mediante tres escenarios de presupuesto —bajo, medio y alto—, todos expresados en pesos chilenos (CLP). El escenario bajo representa el mínimo desembolso posible para un prototipo académico y considera que los cuatro integrantes trabajan sin sueldo. El escenario medio valoriza las horas aportadas por Fernanda Poblete, Catalina Gutiérrez, Benjamín Vallejos y Michael Ibáñez e incorpora los costos directos necesarios para desarrollar y validar Signy en la escala académica actual. El escenario alto representa una evolución hacia un piloto profesional, con mayor dedicación del equipo, servicios pagados adicionales y preparación para una publicación Android. Esta evaluación distingue entre desembolso directo y costo de oportunidad del trabajo del equipo.

Recursos humanos

Rol	Horas estimadas	Valor ref./hora	Costo de oportunidad
Product Owner / Gestión de Proyectos — Fernanda Poblete	110 h	$8.603/h	$946.354
Scrum Master / Development Team — Catalina Gutiérrez	110 h	$9.098/h	$1.000.787
Development Team — Benjamín Vallejos	110 h	$5.769/h	$634.615
Development Team — Michael Ibáñez	110 h	$5.769/h	$634.615

Recursos tecnológicos y operativos

Recurso	Supuesto	Costo estimado
Ionic / Angular / TypeScript / MediaPipe	Licencias abiertas o sin costo incremental	$0 CLP
Git / GitHub	Plan gratuito para el contexto académico	$0 CLP
Supabase Pro	Backend, PostgreSQL, Auth, Storage y Edge Functions	$283.020 CLP/año
Dominio .CL	Identidad web y soporte futuro	$9.990 CLP/año
Validación / apoyo profesional LSCh	4 horas de apoyo profesional	$168.068 CLP
Validación / traslados / materiales	Reserva académica para pruebas y documentación	$50.000 CLP

Costo de oportunidad estimado del equipo en el escenario medio (Año 1): $3.216.372 CLP.

Desembolso directo estimado del escenario medio (Año 1): $561.078 CLP, asociado principalmente a Supabase Pro, dominio .CL, validación profesional de LSCh, incentivos a participantes y logística de pruebas.

Costo económico total de referencia del escenario medio (Año 1): $3.777.450 CLP. Este monto combina el valor económico del trabajo de los cuatro integrantes con los desembolsos directos y no representa que el equipo haya recibido o pagado efectivamente esos sueldos.

Comparación de escenarios y selección del presupuesto

Para seleccionar el presupuesto se compararon tres alternativas. El escenario bajo prioriza el mínimo desembolso y supone que los cuatro integrantes trabajan sin sueldo, por lo que es apropiado como referencia de costo monetario mínimo, pero subestima el valor real del trabajo necesario para construir Signy. El escenario medio valoriza las horas reales del equipo y mantiene los costos tecnológicos y de validación en un nivel coherente con una fase académica. El escenario

alto incorpora mayor dedicación, servicios pagados adicionales y preparación de operación comercial, por lo que resulta sobredimensionado para la etapa actual.

Escenario	Costo Año 1	Costo total 3 años	Adecuación a Signy
Bajo costo	$154.024 CLP	$476.073 CLP	Útil para medir el mínimo desembolso; no valoriza el trabajo del equipo.
Medio costo — seleccionado	$3.777.450 CLP	$6.405.570 CLP	Representa la fase académica actual: equipo de 4, infraestructura real y validación acotada.
Alto costo	$6.959.621 CLP	$13.845.443 CLP	Adecuado para un piloto profesional y futura comercialización; excede las necesidades actuales.

Se selecciona el escenario medio porque es el que mejor equilibra realismo económico y alcance académico. Signy ya cuenta con autenticación, Supabase/PostgreSQL, lecciones, gamificación, perfil, administración, caché y reconocimiento de señas con un clasificador propio, por lo que no corresponde modelarlo como un proyecto de costo mínimo sin valorar el trabajo del equipo. Al mismo tiempo, la fase actual no contempla publicación comercial, soporte permanente ni operación a gran escala; por ello, tampoco resulta justificable asumir los costos completos del escenario alto. El escenario medio reconoce el aporte de los cuatro integrantes, mantiene licencias de software mayoritariamente gratuitas, considera Supabase Pro, dominio, validación profesional y logística de pruebas, y registra la colaboración de la Oficina de Inclusión Social para Personas con Discapacidad de DIDECO Huechuraba como apoyo no monetario con costo $0 CLP.

Viabilidad

El proyecto es viable en contexto académico bajo el escenario medio porque las tecnologías principales utilizadas —Ionic, Angular, TypeScript, Capacitor, MediaPipe, Git y GitHub— no requieren licencias pagadas para el alcance actual; el equipo ya dispone de computadores y dispositivos Android para pruebas; y el gasto directo se concentra en infraestructura, validación y logística. La selección del escenario medio evita subestimar el trabajo necesario y, a la vez, evita incorporar gastos comerciales que todavía están fuera del alcance, como publicación en tiendas, soporte permanente o iOS. La colaboración de DIDECO Huechuraba facilita el contacto con personas sordas para actividades de validación sin representar financiamiento, patrocinio ni ingreso para el proyecto.

L. Despliegue y operación

Arquitectura de despliegue

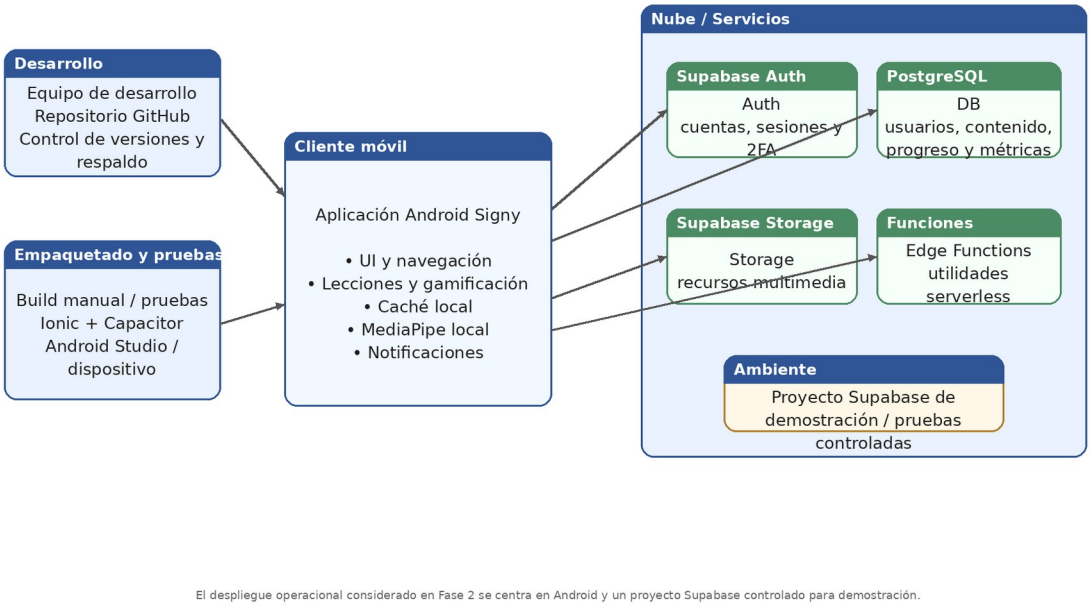


Figura 20. Arquitectura de despliegue y operación de Signy. Fuente: elaboración propia del equipo Signy (2026).

Justificación del alcance Android: el proyecto se valida en Android porque es el entorno sobre el cual ya se ha probado la integración entre Ionic, Capacitor, cámara, notificaciones y caché. Dado el tiempo académico disponible, agregar iOS en esta fase incrementaría el esfuerzo de pruebas, permisos, compatibilidad y publicación. Por ello, Android constituye la plataforma objetivo de la Fase 2 y iOS queda como proyección futura.

Ambientes

1. Desarrollo: ejecución local/web para implementación y depuración rápida.
2. Pruebas Android: emulador o dispositivo físico para cámara, permisos, caché y notificaciones.
3. Demostración: versión estable del Sprint conectada a un proyecto Supabase configurado para la presentación y validación académica.

CI/CD

Con la evidencia disponible en el informe no se documenta un pipeline CI/CD automatizado como requisito actual. El procedimiento mínimo consiste en actualizar la rama de trabajo, ejecutar compilación/pruebas, integrar cambios revisados a la rama principal y generar una versión Android para demostración. Como mejora futura puede incorporarse GitHub Actions para automatizar instalación de dependencias, lint, pruebas y build en cada Pull Request.

Configuración

La solución requiere Node.js/npm, dependencias declaradas por el proyecto, Ionic/Angular, Capacitor y configuración de Supabase. Las URL, claves públicas y demás variables necesarias deben gestionarse en archivos de entorno/configuración no sensibles. Secretos privados, service-role keys y credenciales personales nunca deben quedar expuestos en el repositorio. Para Android se requiere Android Studio/SDK y un emulador o dispositivo compatible. MediaPipe descarga su modelo de manos la primera vez que se usa la cámara, por lo que esa primera práctica necesita internet; el clasificador propio viaja dentro de la app (assets/modelos/clasificador-senas.json). Para reentrenarlo se usa Python 3.10 a 3.13 con las dependencias de ml/requirements.txt.

Manual de instalación/despliegue

1. Clonar o descargar el repositorio autorizado del proyecto.
2. Instalar las dependencias con el gestor definido por el proyecto (npm).
3. Configurar las variables de entorno o archivos de configuración de Supabase sin incluir secretos privados en el repositorio.
4. Ejecutar la aplicación en modo desarrollo y comprobar conexión con Supabase.
5. Para Android, sincronizar el proyecto Capacitor con la plataforma Android y abrirlo en Android Studio.
6. Compilar y ejecutar en emulador o dispositivo físico, concediendo permisos de cámara/notificaciones cuando se soliciten.
7. Ejecutar los casos críticos de prueba antes de utilizar la versión como demostración.
8. Registrar la versión/commit utilizado para que la demostración sea reproducible.

Manual de usuario / administrador

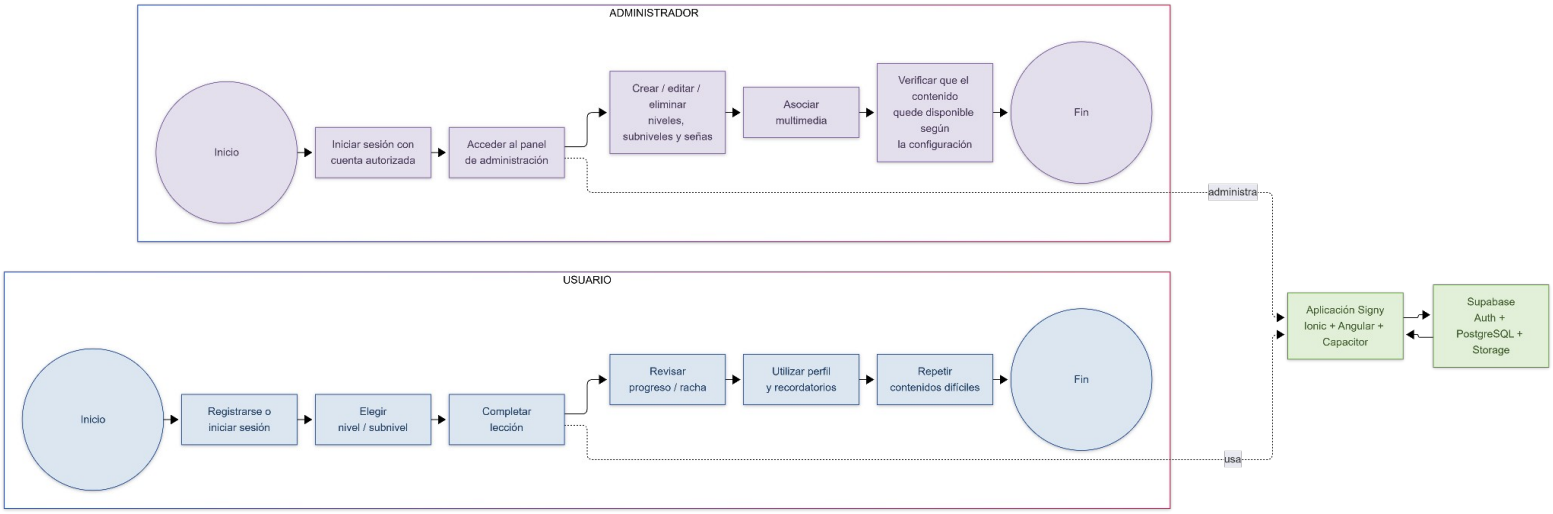


Figura 21. Flujo de usuario y administrador de Signy.

Fuente: elaboración propia del equipo Signy (2026), diagramada con Mermaid, a partir de los flujos funcionales definidos para la aplicación.

Plan básico de continuidad y mantenimiento

1. Mantener respaldos y aprovechar las capacidades de recuperación disponibles en la plataforma de base de datos según el plan contratado.
2. Conservar el repositorio Git como historial de código y etiquetar versiones estables utilizadas en demostraciones.
3. Revisar periódicamente dependencias y actualizar solo después de pruebas de regresión.
4. Monitorear errores de autenticación, consultas, almacenamiento y funciones serverless.
5. Documentar los cambios de esquema y aplicar los scripts SQL de forma controlada.
6. Mantener separadas las configuraciones de desarrollo y demostración/producción.
7. Revisar el contenido LSCh con la comunidad sorda cuando se agreguen nuevas señas, grabar sus muestras en AI Studio y reentrenar el clasificador con ml/entrenar.py.

M. Resultados del proyecto

Producto final / estado actual

Al 8 de noviembre de 2026, Signy es una aplicación Android completa: autenticación y seguridad de cuenta, persistencia real en Supabase, ruta de aprendizaje con 4 niveles y 12 señas validadas con la comunidad sorda, lecciones interactivas, gamificación con logros y ranking, perfil social, administración de contenido, AI Studio, Dashboard de KPIs, notificaciones, caché multimedia y reconocimiento de señas con MediaPipe y un clasificador propio integrado en la práctica de la lección.

Resultados obtenidos

1. Se reemplazó la dependencia de almacenamiento local por una capa de datos real en Supabase para contenido y progreso.
2. Se consolidó un flujo de aprendizaje estructurado y gamificado que permite avanzar por niveles y subniveles.
3. Se incorporaron mecanismos de motivación como vidas, racha, XP y recordatorios.
4. Se habilitó una administración de contenido que reduce la necesidad de intervenir manualmente la base de datos.
5. Se optimizó el uso de multimedia mediante caché y precarga, mejorando la experiencia repetida de una lección.
6. Se integró el reconocimiento de señas con MediaPipe en la lección y se entrenó un clasificador propio con TensorFlow, a partir de un dataset grabado en el AI Studio de la app.
7. Se estructuró la gestión del proyecto mediante Product Backlog, cinco Sprints, cinco Releases, criterios de aceptación, plan de pruebas y control de alcance.
8. Se verificó la calidad con 664 pruebas unitarias de la aplicación y 69 pruebas del pipeline de aprendizaje automático, todas aprobadas, y se corrigieron trece defectos, dos de ellos de seguridad.
9. Se validaron las señas con personas de la comunidad sorda y la usabilidad con personas externas al equipo.

Indicadores y métricas

Indicador	Valor / estado	Interpretación
Entregables definidos	11	Permite medir el cumplimiento del alcance comprometido.
Entregables implementados	11 de 11	Todos los entregables comprometidos.
Entregables con validación externa	Realizada	Señas validadas con la comunidad sorda; usabilidad probada con personas externas.
Historias de usuario terminadas	20 de 20	HU-20 se cerró el 26/10/2026.
Pruebas automatizadas	733 (664 app + 69 ML)	Todas aprobadas; 1 prueba omitida por cuarentena.
Cobertura de sentencias	79,09 %	Cobertura de líneas: 80,88 %.
Tablas de la base de datos	16 + auth.users	Modelo relacional con RLS en todas las tablas.
Commits en el repositorio	97 en 4 ramas	Trazabilidad entre historias y código.
Defectos registrados	14 (13 cerrados)	Dos defectos de seguridad corregidos; el abierto no afecta la app.
Sprints / Releases planificados	5 / 5	Los cinco Sprints completados.

Indicador	Valor / estado	Interpretación
Exactitud del clasificador	70,5 % (90,3 % sobre el umbral)	Con muestras de prueba; con personas nuevas baja a 27,8 %: falta ampliar el dataset.

Proyección

Las prioridades siguientes son grabar muestras de más personas para mejorar la exactitud del clasificador con usuarios nuevos, ampliar el vocabulario validado con la comunidad sorda y mantener una versión estable de demostración. En una etapa futura podrían evaluarse la publicación en Play Store, el soporte para iOS, el reconocimiento de expresiones no manuales y una mayor cobertura sin conexión.

Conclusión

Signy aborda una necesidad real: la escasez de recursos pedagógicos, estructurados y tecnológicamente actuales para aprender Lengua de Señas Chilena. A lo largo del informe se definió el problema, se delimitó el producto, se especificaron requisitos, se diseñó la arquitectura y se estructuró la gestión del proyecto mediante Scrum, backlog, releases, sprints, pruebas, validación y lineamientos de despliegue. En esta Fase 2 puede afirmarse que los objetivos de análisis, definición, planificación y construcción se cumplen satisfactoriamente, y que además existe una base funcional relevante del producto, con autenticación, estructura de aprendizaje, gamificación, gestión de contenido, perfil, recursos multimedia y reconocimiento de señas con MediaPipe Hands y un clasificador propio integrado en la lección, con señas validadas junto a la comunidad sorda.

El principal aporte del proyecto consiste en proponer una plataforma móvil enfocada específicamente en LSCh, con una mirada que combina accesibilidad, aprendizaje autónomo y gestión de producto. También queda claro que la continuidad del proyecto requiere ampliar el vocabulario validado con la comunidad sorda, aumentar el dataset con más personas para mejorar el reconocimiento, ampliar las pruebas con usuarios y fortalecer la operación del producto si en el futuro se quisiera evolucionar desde un prototipo académico hacia una solución de uso real. Como proyección, Signy posee potencial para transformarse en una herramienta de apoyo significativa para el aprendizaje de LSCh en Chile, siempre que su crecimiento mantenga el foco en la calidad pedagógica, la validación con referentes y una gestión ágil disciplinada.

Individual Conclusions

Written in English, as required by the evaluation guidelines for Phase 2.

Fernanda Poblete – Product Owner

Signy shows that a clear product vision is as important as good code. As Product Owner, I organised the Product Backlog into 20 user stories, prioritised them by value and risk, and recorded every scope change in the change log. These decisions helped the team finish all 20 stories and all 11 deliverables without losing the main goal: helping hearing people learn Chilean Sign Language. The validation with Deaf people, coordinated with DIDECO Huechuraba, was completed; the main difficulty was the slow communication to schedule the meetings.

Catalina Gutiérrez – Scrum Master and Development Team

In this phase Scrum helped us turn a large and uncertain project into small and measurable increments. Splitting the old Sprint 4 into two Sprints separated the technical risk of MediaPipe from the external validation, and this decision protected the rest of the plan. I also contributed to the application itself, especially authentication, password recovery, the quiz, media caching and the onboarding. My conclusion is that a well-facilitated process makes problems visible early, but we still need to record impediments and Sprint metrics more consistently.

Benjamín Vallejos – Development Team

Signy is now a real product that people can use, not only a design. I worked on the user experience, the social profile, two-factor authentication, the admin panel, notifications, gamification and the third version of the sign recognition engine, and I wrote the automated test suite. Having 664 passing unit tests gave the team confidence to change important parts of the code without breaking what already worked. I conclude that quality and user experience must grow together: a beautiful screen is not enough if the data behind it is not secure and reliable.

Michael Ibáñez – Development Team

The most challenging part of Signy was sign recognition. I built the first integration of MediaPipe, the AI Studio and the DTW comparison that replaced the pixel heuristic inside the lesson, and I worked on the neural network that finally replaced DTW because it was more accurate and lighter for the phone. This work showed me that detecting the 21 points of a hand is not the same as recognising a sign correctly: the system needs normalisation, many samples from different people and, above all, linguistic validation. My conclusion is that the technical base is ready, and the next step is to add samples from many more people.

Individual Reflections

Written in English, as required by the evaluation guidelines for Phase 2.

Fernanda Poblete

This phase confirmed my interest in project management, Product Ownership and PMO roles. I learned to transform technical information into clear decisions and deliverables for the team and the teacher. My main area for improvement is making decisions faster: sometimes I review a topic too much before closing it. In the next phase I want to decide with the evidence available, write down the criteria I used and adjust later if needed. I also learned that managing a technological product also means managing people, expectations and time.

Catalina Gutiérrez

I learned that my analytical skills are most useful when I do not try to control every detail. In Phase 1 I identified that I needed to delegate more, and in this phase I practised it by dividing big problems into small tasks with clear owners and criteria. I still need to avoid taking too many tasks at the same time, because it reduces my ability to see the whole project. Professionally, this experience strengthened my interest in technology management, risk management and continuous improvement.

Benjamín Vallejos

This phase helped me connect design with the real logic of the system. I enjoyed making the application clear and attractive, but I also realised that I need a deeper understanding of the backend and data persistence, not to specialise in it, but to estimate better and to anticipate dependencies. I

also learned to communicate earlier with Michael and the rest of the team when a screen depends on data that is not ready. In the future I see myself in roles that combine analysis, data and user experience.

Michael Ibáñez

From a technical point of view I am satisfied with my work on architecture, Supabase, persistence and MediaPipe, but this phase also made me face a weakness I had identified before: documentation and communication. I improved by explaining my technical decisions and leaving more traceability, but I still need to document changes at the same time I implement them and to announce any change that may affect other modules. I want to continue growing in software architecture, backend, cloud and artificial intelligence.

Referencias bibliográficas

Las referencias se presentan siguiendo el formato APA (7.ª edición). La fuente de cada figura se indica además bajo la figura correspondiente.

Marco legal y estadístico

Biblioteca del Congreso Nacional de Chile. (2021). Ley N.° 21.303. Modifica la ley N.° 20.422, que establece normas sobre igualdad de oportunidades e inclusión social de personas con discapacidad, para promover el uso de la lengua de señas. Diario Oficial de la República de Chile, 22 de enero de 2021. https://www.bcn.cl/leychile/navegar?idNorma=1154963

Biblioteca del Congreso Nacional de Chile. (2010). Ley N.° 20.422. Establece normas sobre igualdad de oportunidades e inclusión social de personas con discapacidad. https://www.bcn.cl/leychile/navegar?idNorma=1010903

Ministerio de Desarrollo Social y Familia, Instituto Nacional de Estadísticas y Servicio Nacional de la Discapacidad. (2016). Segundo Estudio Nacional de la Discapacidad (ENDISC II, 2015). SENADIS. https://www.senadis.gob.cl/pag/355/1197/ii_estudio_nacional_de_discapacidad

Servicio Nacional de la Discapacidad. (2021). SENADIS destaca ley que reconoce a la lengua de señas como lengua oficial de las personas sordas [comunicado de prensa]. https://www.senadis.gob.cl/sala_prensa/d/noticias/8472

Referentes y benchmarking

Acuña Robertson, X., Adamo Quintela, D. y Cabrera Ramírez, I. (2009). Diccionario bilingüe lengua de señas chilena-español (Vols. 1–2). Ministerio de Educación de Chile.

Duolingo, Inc. (s. f.). Duolingo. Recuperado el 8 de noviembre de 2026, de https://www.duolingo.com

SignSchool Technologies LLC. (s. f.). SignSchool. Recuperado el 8 de noviembre de 2026, de https://www.signschool.com

Spread the Sign. (s. f.). Spread the Sign. Recuperado el 8 de noviembre de 2026, de https://www.spreadthesign.com

Documentación técnica

Google. (s. f.-a). Angular documentation. Recuperado el 8 de noviembre de 2026, de https://angular.dev

Google. (s. f.-b). Hand Landmarker. Google AI Edge. Recuperado el 8 de noviembre de 2026, de https://developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker

Ionic. (s. f.-a). Capacitor documentation. Recuperado el 8 de noviembre de 2026, de https://capacitorjs.com/docs

Ionic. (s. f.-b). Ionic Framework documentation. Recuperado el 8 de noviembre de 2026, de https://ionicframework.com/docs

Supabase Inc. (s. f.). Supabase documentation. Recuperado el 8 de noviembre de 2026, de https://supabase.com/docs

three.js authors. (s. f.). three.js documentation. Recuperado el 8 de noviembre de 2026, de https://threejs.org/docs

Metodología, arquitectura y algoritmos

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., Wicke, M., Yu, Y. y Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. En 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16) (pp. 265–283). USENIX Association. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi

Adzic, G. (2012). Impact mapping: Making a big impact with software products and projects. Provoking Thoughts.

Kruchten, P. B. (1995). The 4+1 view model of architecture. IEEE Software, 12(6), 42–50. https://doi.org/10.1109/52.469759

Patton, J. (2014). User story mapping: Discover the whole story, build the right product. O'Reilly Media.

Sakoe, H. y Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. IEEE Transactions on Acoustics, Speech, and Signal Processing, 26(1), 43–49. https://doi.org/10.1109/TASSP.1978.1163055

Schwaber, K. y Sutherland, J. (2020). La Guía de Scrum: La guía definitiva de Scrum: Las reglas del juego. https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Spanish-Latin-South-American.pdf

Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L. y Grundmann, M. (2020). MediaPipe Hands: On-device real-time hand tracking. arXiv. https://doi.org/10.48550/arXiv.2006.10214

Fuente del proyecto

Vallejos, B., Gutiérrez, C., Poblete, F. e Ibáñez, M. (2026). Proyecto-Senias [repositorio de software]. GitHub. https://github.com/RatalinaPrograma/Proyecto-Senias

Fuente de las figuras y de los recursos gráficos

Figura 1. Diagrama AS IS del proceso de aprendizaje de LSCh antes de Signy. Elaboración propia del equipo Signy (2026).

Figura 2. Diagrama TO BE del proceso de aprendizaje propuesto para Signy. Elaboración propia del equipo Signy (2026).

Figura 3. Mapa mental del problema abordado por Signy: dificultades detectadas, impactos y solución propuesta. Elaboración propia del equipo Signy (2026).

Figura 4. Mapa de actores de Signy, desde el usuario hasta las administraciones públicas vinculadas a la inclusión de personas sordas en Chile. Elaboración propia del equipo Signy (2026).

Figura 5. Diagrama lógico de arquitectura de Signy. Elaboración propia del equipo Signy (2026).

Figura 6. Vistas 4+1 de la arquitectura de Signy. Elaboración propia del equipo Signy (2026).

Figura 7. Modelo de datos base de Signy. Elaboración propia del equipo Signy (2026).

Figura 8. Diagrama de integraciones de Signy. Elaboración propia del equipo Signy (2026).

Figura 9. Flujo del componente de IA / visión computacional. Elaboración propia del equipo Signy (2026).

Figura 10. Product Backlog priorizado de Signy (20 historias). Elaboración propia del equipo Signy (2026).

Figura 11. User Story Mapping de Signy. Elaboración propia del equipo Signy (2026).

Figura 12. Release Planning de Signy (5 releases). Elaboración propia del equipo Signy (2026).

Figura 13. Roadmap de Sprints propuesto para Signy. Elaboración propia del equipo Signy (2026).

Figura 14. Release burndown de Signy (20 historias de usuario). Elaboración propia del equipo Signy (2026).

Figura 15. Scrum Board – Sprint 1: Base, seguridad y datos. Elaboración propia del equipo Signy (2026).

Figura 16. Scrum Board – Sprint 2: Aprendizaje principal. Elaboración propia del equipo Signy (2026).

Figura 17. Scrum Board – Sprint 3: Experiencia, comunidad y administración. Elaboración propia del equipo Signy (2026).

Figura 18. Scrum Board – Sprint 4: Integración MediaPipe. Elaboración propia del equipo Signy (2026).

Figura 19. Scrum Board – Sprint 5: Validación, personalización y cierre. Elaboración propia del equipo Signy (2026).
Figura 20. Arquitectura de despliegue y operación de Signy. Elaboración propia del equipo Signy (2026).
Figura 21. Flujo de usuario y administrador de Signy. Elaboración propia del equipo Signy (2026).
Figura 22. Resumen de las pruebas unitarias de la aplicación. Captura de pantalla del equipo Signy (2026).
Figura 23. Pruebas de la aplicación en Karma y Jasmine. Captura de pantalla del equipo Signy (2026).
Figura 24. Pruebas del pipeline de inteligencia artificial. Captura de pantalla del equipo Signy (2026).
Figura 25. Práctica con cámara con la seña reconocida al 100 %. Captura de pantalla del equipo Signy (2026).
Figura 26. Tablas de la base de datos en Supabase. Captura de pantalla del equipo Signy (2026).
Figura 27. Edge Functions desplegadas en Supabase. Captura de pantalla del equipo Signy (2026).
Figura 28. Buckets de Supabase Storage. Captura de pantalla del equipo Signy (2026).
Figura 29. Aportes al repositorio por integrante. Captura de pantalla del equipo Signy (2026).

Logotipo institucional Duoc UC (portada y encabezado). Duoc UC. Plantilla institucional «Formato Guía» entregada para la asignatura Portafolio de Título (APT122). Uso académico interno.

Íconos de interfaz de la aplicación. Ionicons (Ionic), licencia MIT. https://ionic.io/ionicons

Tipografías Baloo 2 y Manrope utilizadas en la aplicación. Google Fonts, bajo licencia SIL Open Font License 1.1. https://fonts.google.com

Ilustraciones del onboarding de la aplicación (AprendeSeñas, AprendeJugando, EstasListo y recuperacuenta). Imágenes generadas con inteligencia artificial mediante ChatGPT (OpenAI), a partir de indicaciones redactadas por el equipo Signy (2026). https://chatgpt.com

Anexo de evidencias

Este anexo reúne capturas tomadas por el equipo el 8 de noviembre de 2026 en el computador de desarrollo (Windows), en la aplicación, en el panel de Supabase y en GitHub. Respaldan los resultados de las secciones I (pruebas), F (componente de IA), E (modelo de datos e integraciones), G (trabajo en equipo) y M (resultados). Las mismas imágenes están en la carpeta Documentos/FASE 2/Evidencias Proyecto/Evidencias de sistema del repositorio.

A. Pruebas automatizadas

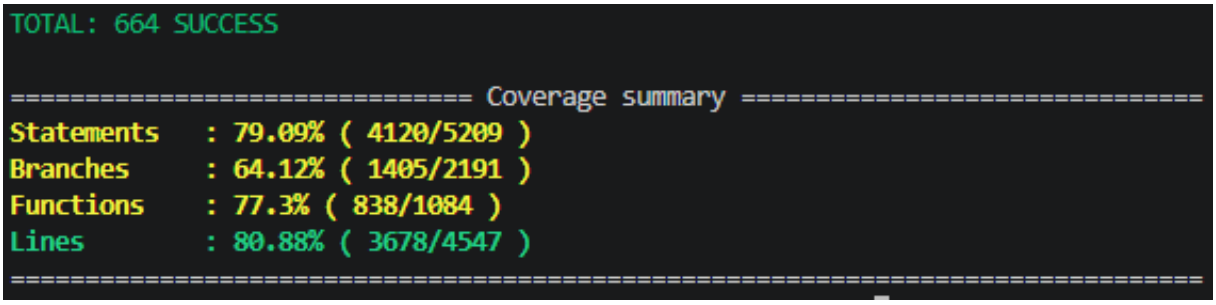


Figura 22. Resumen de la ejecución de las 664 pruebas unitarias de la aplicación con su cobertura (ng test --code-coverage): 664 aprobadas y 79,09 % de cobertura de sentencias. Fuente: captura de pantalla del equipo Signy (2026).

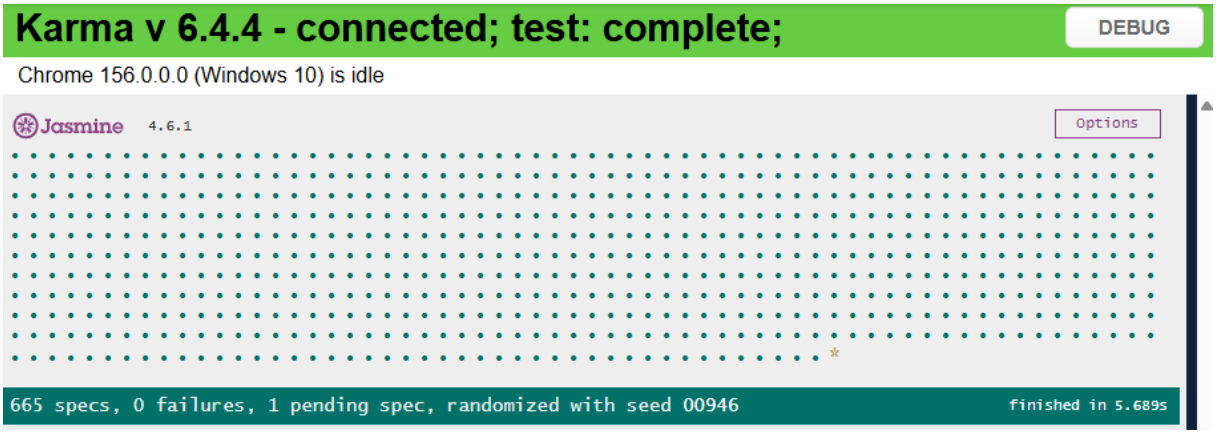


Figura 23. Ejecución de las pruebas de la aplicación en Karma y Jasmine (Chrome, Windows): 665 especificaciones, 0 fallas y 1 pendiente (PracticaPage, en cuarentena). Fuente: captura de pantalla del equipo Signy (2026).

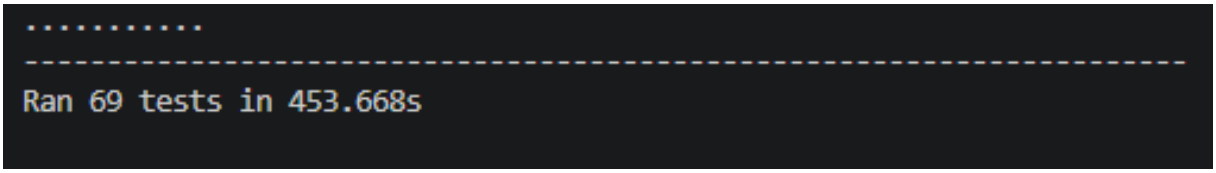


Figura 24. Ejecución de las 69 pruebas del pipeline de inteligencia artificial (python -m unittest discover -s ml/tests). Fuente: captura de pantalla del equipo Signy (2026).

B. Aplicación funcionando

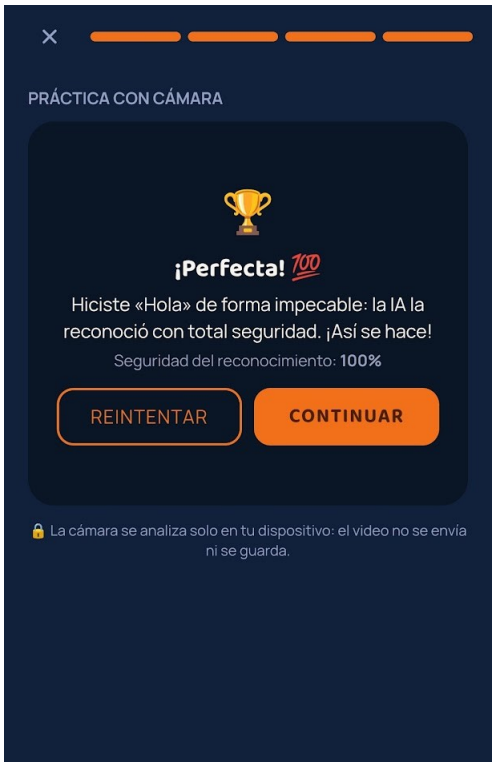


Figura 25. Práctica con cámara: la seña «Hola» se reconoce con 100 % de seguridad y la app muestra la felicitación, sin consejos. Fuente: captura de pantalla del equipo Signy (2026).

C. Base de datos y backend (Supabase)

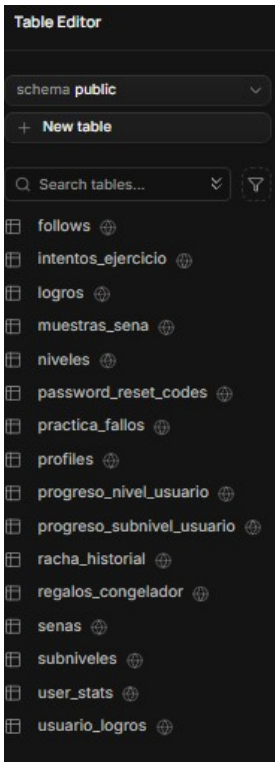


Figura 26. Tablas del esquema public en el Table Editor de Supabase (16 tablas). Fuente: captura de pantalla del equipo Signy (2026).

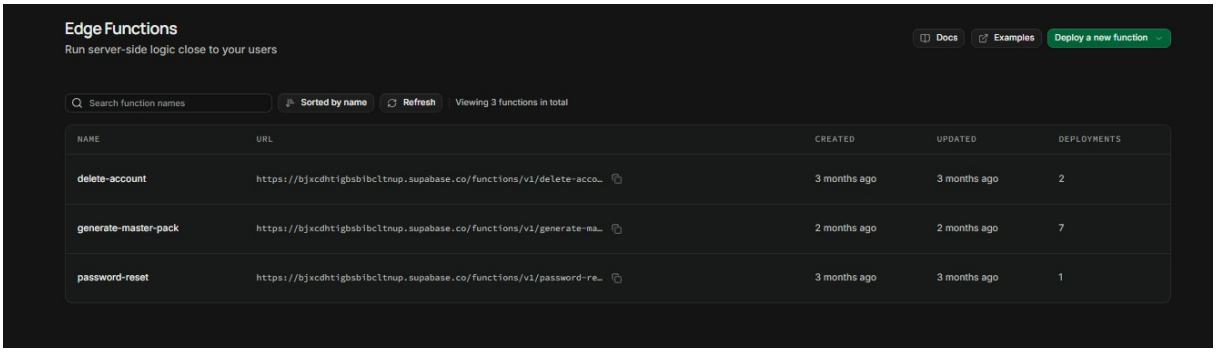


Figura 27. Edge Functions desplegadas en Supabase. La app usa delete-account y password-reset; generate-master-pack es una prueba de la rama dev (empaquetar todo el contenido en un solo archivo) que la versión final no utiliza. Fuente: captura de pantalla del equipo Signy (2026).

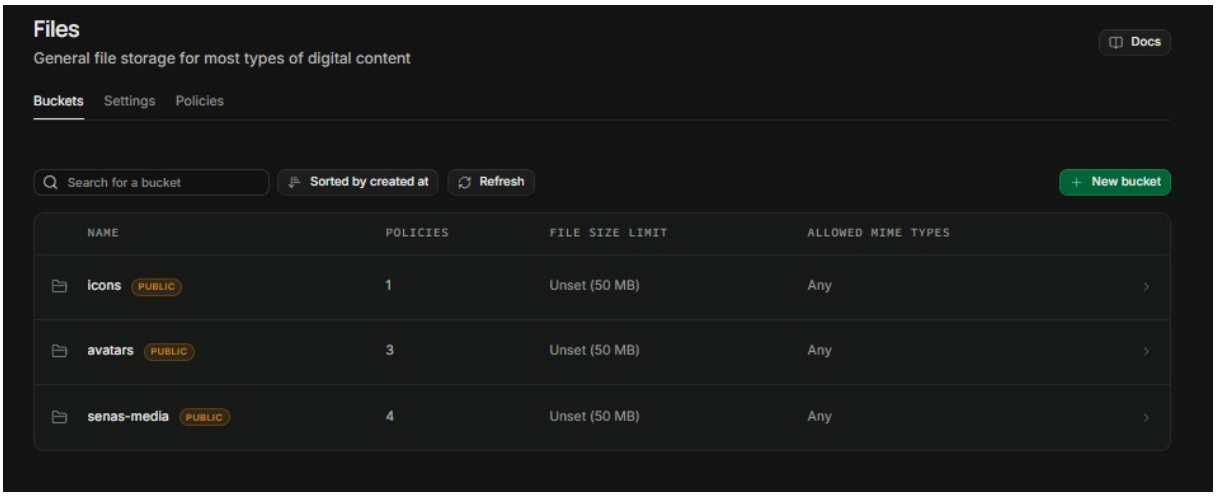


Figura 28. Buckets de Supabase Storage: icons (íconos de la interfaz), avatars (fotos de perfil) y senas-media (videos de las señas), cada uno con sus políticas de acceso. Fuente: captura de pantalla del equipo Signy (2026).

D. Repositorio y trabajo en equipo



Figura 29. Aportes al repositorio en GitHub (Insights, Contributors) entre agosto y noviembre de 2026. benjaavmm corresponde a Benjamín Vallejos, RatalinaPrograma a Catalina Gutiérrez, Nachooop17 a Michael Ibáñez y FernandaPoblete a Fernanda Poblete; claude corresponde a los commits hechos con el asistente de IA Claude. Fuente: captura de pantalla del equipo Signy (2026).